

Politechnika Wrocławska

Wydział Informatyki i Telekomunikacji

Kierunek: **Informatyka Algorytmiczna (INA)**

PRACA DYPLOMOWA MAGISTERSKA

**Analiza wybranych metod detekcji halucynacji w dużych
modelach językowych**

Aleksandra Wójcik

Opiekun pracy:
dr hab. inż. Łukasz Krzywiecki, prof. uczelni

Słowa kluczowe: *Duże modele językowe, halucynacje*

Wrocław 2026

Streszczenie

Modele językowe oparte na architekturze transformera mogą generować odpowiedzi płynne językowo, lecz niezgodne z faktami lub z dostarczoną referencją. Detekcja takich odpowiedzi jest trudna, ponieważ sama płynność tekstu nie stanowi kryterium poprawności. Niniejsza praca bada hipotezę, że odpowiedzi o niskiej zgodności z referencją pozostawiają mierzalne ślady w wewnętrznej reprezentacji modelu, tj. w stanach ukrytych oraz rozkładach prawdopodobieństwa kolejnych tokenów. Metoda bazuje na ekstrakcji ośmiu interpretowalnych cech, z których część stanowi oryginalne propozycje autora. Obejmują one dryf reprezentacji między warstwami i między tokenami, entropię, niepewność modelu względem najlepszego tokenu oraz ich iloczyny z odpowiednimi dryfami, a także odległość tokenu od lokalnego centroidu kontekstu i rozproszenie kandydatów w przestrzeni odosadzania. Cechy te wyznaczone są dla każdego tokenu wygenerowanej odpowiedzi i każdej warstwy, tworząc macierz o zmiennej liczbie wierszy zależnej od długości odpowiedzi. Aby uzyskać stałowymiarową reprezentację, każda kolumna macierzy, odpowiadająca jednej cesze, jest agregowana przez obliczenie średniej, odchylenia standardowego oraz maksimum, co daje wektor przekazywany do klasyfikatora. Eksperymenty przeprowadzono na modelu Llama-3.1-8B na zbiorach pytań i odpowiedzi TriviaQA, SQuAD oraz NQOpen. Poprawność odpowiedzi wyznaczono automatycznie przy użyciu metryk BLEURT i ROUGE-L względem odpowiedzi referencyjnych. Etykiety mają charakter pseudoetykiet i stanowią przybliżenie poprawności. Najlepszy wynik uzyskano na zbiorze TriviaQA, dla którego AUROC wyniósł 0,897. Wyniki bez transferu między zbiorami mieszczą się w przedziale od 0,831 do 0,897, co wskazuje, że badane cechy niosą użyteczny sygnał predykcyjny. Test generalizacji między zbiorami potwierdza względną stabilność klasyfikatora, jednak w niektórych konfiguracjach wyniki transferu między zbiorami spadają do 0,746. Wyniki są konkurencyjne względem wybranych metod białoskrzynkowych opartych na cechach jawnych modelu.

Abstract

Large language models based on the transformer architecture can generate fluent but factually incorrect responses or responses inconsistent with a provided reference. Detecting such outputs is challenging because linguistic fluency alone does not indicate correctness. This thesis investigates the hypothesis that responses with low agreement with a reference leave measurable traces in the model's internal representation, specifically in hidden states and next-token probability distributions. The proposed method extracts eight interpretable features. These include layer-wise and token-to-token representation drift, entropy, the model's uncertainty with respect to the top token, their products with the respective drifts, the cosine distance of a token from the local context centroid, and the dispersion of top candidates in the unembedding space. Features are computed for each token of the generated answer and each layer, forming a matrix whose number of rows varies with answer length. To obtain a fixed-size representation, each column of the matrix, corresponding to a single feature, is summarised by its mean, standard deviation, and maximum, producing a vector passed to the classifier.

Experiments were conducted on Llama-3.1-8B using the TriviaQA, SQuAD, and NQOpen question-answering datasets. Answer correctness was determined automatically with BLEURT and ROUGE-L against reference answers. These labels are pseudo-labels approximating correctness. The best result was obtained on TriviaQA, achieving AUROC = 0,897. Results without cross-dataset transfer range from 0,831 to 0,897, indicating that the investigated features carry a useful predictive signal. A cross-dataset generalisation test confirms relative stability of the classifier, though in some configurations cross-dataset transfer results drop to 0,746. The results are competitive with selected white-box hallucination detection methods based on explicit model features.

Spis treści

1	Wstęp	1
1.1	Cel i zakres pracy	1
1.2	Pytania badawcze	2
1.3	Struktura pracy	2
2	Problem halucynacji w dużych modelach językowych	3
2.1	Architektura Transformer	3
2.1.1	Tokenizacja i osadzenia	4
2.1.2	Warstwy Transformera	4
2.1.3	Mechanizm uwagi	5
2.1.4	Warstwa propagacji w przód	6
2.1.5	Normalizacja warstw	7
2.1.6	Strumień residualny	8
2.1.7	Stany ukryte	8
2.1.8	Dekodowanie	9
2.1.9	Model Llama-3.1-8B	9
2.2	Charakterystyka i detekcja halucynacji	9
2.2.1	Definicja	9
2.2.2	Pseudohalucynacje	10
2.2.3	Rodziny metod detekcji halucynacji	10
2.2.4	Pozycjonowanie niniejszej pracy	12
3	Metodologia badań	13
3.1	Potok badawczy	13
3.2	Pozyskiwanie i przetwarzanie danych	13
3.2.1	Generowanie odpowiedzi	13
3.2.2	Etykietowanie	14
3.2.3	Ograniczenia etykietowania	15
3.2.4	Ekstrakcja macierzy cech	15
3.2.5	Zagregowany wektor cech	15
3.2.6	Skalowanie cech	16
3.3	Badane cechy	17
3.3.1	Formalizmy	17
3.3.2	Definicje pomocnicze	18
3.3.3	Dryf międzywarstwowy	19
3.3.4	Dryf tokena	19
3.3.5	Entropia	20
3.3.6	Niepewność	21
3.3.7	Odległość kontekstowa	21

3.3.8	Rozproszenie predykcyjne	22
3.3.9	Cechy złożone	23
3.3.10	Wybór statystyk	24
3.3.11	Podsumowanie cech	24
3.4	Klasyfikator	25
3.4.1	Obsługa nierówności klas w klasyfikatorach	25
3.4.2	Regresja logistyczna	25
3.4.3	Protokół treningu i walidacji	26
4	Wyniki badań i ich analiza	28
4.1	Opis eksperymentu	28
4.2	Wyniki klasyfikacji	28
4.3	Porównanie skuteczności z innymi metodami białoskrzynkowymi	29
4.4	Badanie skuteczności metody na danych z różnych zbiorów danych	30
4.5	Analiza cech	30
4.5.1	Wnioski	31
4.6	Analiza warstwowa	32
4.6.1	Analiza cech bazowych	32
4.7	Podsumowanie wyników	34
5	Podsumowanie	36
5.1	Realizacja celów pracy	36
5.2	Odpowiedzi na pytania badawcze	36
5.3	Ograniczenia	37
5.4	Kierunki dalszych badań	37
	Spis literatury	38
	Dodatki	
A	Materiały uzupełniające i deklaracje	42
A.1	Struktura promptów i charakterystyka zbiorów danych	42
A.1.1	Zbiór TriviaQA	42
A.1.2	Zbiór SQuAD	42
A.1.3	Zbiór NQOpen	43
A.2	Dobór parametrów wewnętrznych cech	43
A.2.1	Rozmiar okna kontekstowego	43
A.2.2	Rozmiar zbioru kandydatów	43
A.3	Deklaracja wykorzystania sztucznej inteligencji	44

Rozdział 1

Wstęp

Generatywne modele językowe stają się coraz powszechniej stosowanym narzędziem w systemach wspomagania decyzji, edukacji, medycynie i prawie. Ich użyteczność zależy jednak od poprawności generowanych odpowiedzi. Opisywane modele mogą produkować tekst brzmiący przekonująco, ale zawierający informacje niezgodne z faktami. Zjawisko to, nazywane halucynacją, stanowi znaczącą przeszkodę w zastosowaniach wymagających wysokiej wiarygodności. Detekcja halucynacji napotyka jednak poważne wyzwania. Poprawność gramatyczna czy płynność narracji w żaden sposób nie świadczą o prawdziwości tekstu. Dotychczasowe próby rozwiązania tego problemu koncentrowały się głównie na weryfikacji spójności wielokrotnych generacji (metody czarnoskrzynkowe) lub na porównywaniu wyników z zewnętrznymi bazami wiedzy. Podejścia te są jednak kosztowne obliczeniowo lub wymagają dostępu do zewnętrznych źródeł danych w czasie rzeczywistym. Alternatywą stają się metody białoskrzynkowe, które poszukują sygnałów świadczących o poprawności bezpośrednio w wewnętrznych stanach i aktywacjach modelu. W tym obszarze wciąż otwarte pozostaje jednak pytanie, które konkretne, interpretowalne cechy wewnętrzne niosą ze sobą informacje pozwalające odróżnić poprawne odpowiedzi od halucynacji.

1.1. Cel i zakres pracy

Niniejsza praca bada hipotezę, która zakłada, że odpowiedzi nieprawdziwe, niezgodne z zewnętrznymi źródłami wiedzy pozostawiają mierzalne ślady w wewnętrznej reprezentacji modelu językowego. Ślady te mogą być uchwycone przez interpretowalne cechy wyekstrahowane z wnętrza modelu. Jeżeli cechy te rzeczywiście różnią się między odpowiedziami zgodnymi a niezgodnymi z referencją, można ich użyć do budowy klasyfikatora wykrywającego halucynacje bez odwoływania się do zewnętrznej bazy wiedzy. Celem pracy jest opracowanie potoku badawczego ekstrakcji interpretowalnych cech wewnętrznych z modelu językowego oraz zbadanie, czy cechy oparte na logitach i stanach ukrytych pozwalają odróżniać odpowiedzi zgodne z referencją od odpowiedzi niezgodnych. Praca ocenia wkład poszczególnych cech w wynik klasyfikacji oraz sprawdza, czy wyuczony klasyfikator zachowuje skuteczność przy zmianie zbioru danych. Badania przedstawione w niniejszej pracy wykonane zostały na modelu Llama-3.1-8B [1]. Analizowanymi danymi są cechy wyekstrahowane z sekwencji wygenerowanych przez model w odpowiedzi na pytania ze zbiorów TriviaQA [2], SQuAD [3] i NQOpen [4]. Poprawność generowanych tekstów oceniana jest poprzez porównanie ich z odpowiedziami referencyjnymi zawartymi w tych zbiorach, przy użyciu metod BLEURT [5] oraz ROUGE-L [6].

1.2. Pytania badawcze

Problemem badawczym postawionym w niniejszej pracy jest detekcja odpowiedzi niezgodnych z referencją w zadaniach odpowiadania na pytania, z wykorzystaniem sygnałów wewnętrznych modelu językowego. Dodatkowo w pracy postawiono trzy główne pytania badawcze:

- Q1.** Czy na podstawie cech wewnętrznych modelu można skutecznie rozróżnić odpowiedzi zgodne od niezgodnych z referencją?
- Q2.** Które spośród badanych cech wnoszą największy wkład w wynik klasyfikacji?
- Q3.** Czy proponowana metoda zachowuje swoją skuteczność przy przeniesieniu na inny zbiór danych?

1.3. Struktura pracy

Niniejsza praca została podzielona na logicznie powiązane ze sobą rozdziały, które prowadzą od teoretycznych podstaw analizowanego zjawiska, przez opis zastosowanej metodologii, aż do prezentacji i dyskusji uzyskanych wyników eksperymentalnych. Rozdział pierwszy stanowi wstęp do pracy. Określono w nim cel główny oraz zakres podjętych badań, a także sformułowano pytania badawcze, na które odpowiedzi poszukiwano w toku realizacji eksperymentów. W rozdziale drugim dokonano formalnego postawienia problemu badawczego, którym jest zjawisko halucynacji w dużych modelach językowych. W pierwszej kolejności szczegółowo omówiono architekturę Transformer, stanowiącą fundament współczesnych modeli generatywnych. Następnie przedstawiono definicje, źródła oraz szczegółową charakterystykę zjawiska halucynacji w tych systemach. Rozdział trzeci został w całości poświęcony metodologii przeprowadzonych badań. Opisano w nim kompletny potok badawczy (ang. *pipeline*), służący do ekstrakcji i przetwarzania danych. Przedstawiono badane cechy, scharakteryzowano specyfikę danych wyekstrahowanych z modelu językowego, proces ich etykietowania, a także strukturę i parametry klasyfikatora wykorzystanego do detekcji pseudohalucynacji. Rozdział czwarty zawiera prezentację uzyskanych wyników badań oraz ich pogłębioną analizę merytoryczną. Dokonano w nim analizy skuteczności klasyfikatora na podstawie wybranych miar jakości oraz sformułowano odpowiedzi na postawione we wstępie pytania badawcze. Pracę kończy rozdział piąty, stanowiący podsumowanie całości zrealizowanych badań, w którym wskazano ograniczenia zastosowanego podejścia oraz potencjalne kierunki dalszego rozwoju analizowanej tematyki. Do pracy dołączono również załączniki zawierające m.in. strukturę użytych promptów oraz deklarację dotyczącą wykorzystania narzędzi generatywnej sztucznej inteligencji w procesie przygotowywania tekstu.

Rozdział 2

Problem halucynacji w dużych modelach językowych

Niniejszy rozdział jest poświęcony szczegółowej analizie zjawiska halucynacji w dużych modelach językowych oraz metodom ich detekcji, ze szczególnym uwzględnieniem sygnałów wewnętrznych sieci. Współczesne systemy generowania tekstu, oparte w przeważającej mierze na architekturze Transformer, osiągają doskonałe wyniki w przetwarzaniu języka naturalnego, jednak ich autoregresyjna natura niesie za sobą fundamentalne ryzyko generowania nieprawdziwych treści.

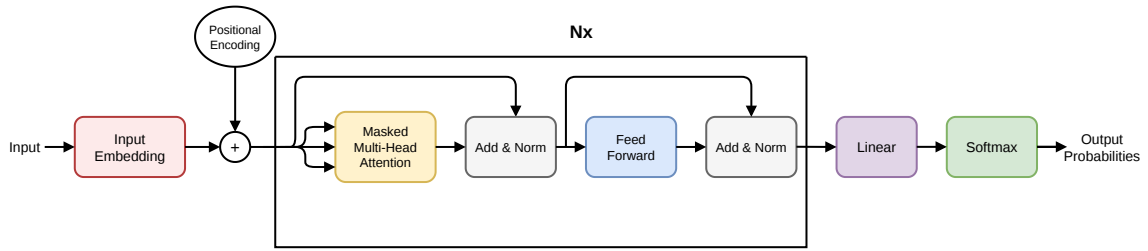
Halucynacja w kontekście dużych modeli językowych (ang. *Large Language Models*) definiowana jest jako proces, w którym model produkuje tekst płynny gramatycznie i przekonujący semantycznie, lecz niezgodny z faktami obiektywnymi, dostarczonym kontekstem źródłowym lub własną strukturą logiczną wypowiedzi. Zjawisko to stanowi kluczową barierę wdrożeniową dla systemów sztucznej inteligencji w domenach krytycznych, takich jak medycyna, prawo czy finanse.

Zrozumienie mechanizmu powstawania tych błędów wymaga głębokiej analizy sposobu, w jaki informacja jest przetwarzana wewnątrz sieci. W związku z tym, w pierwszej części rozdziału szczegółowo opisano architekturę Transformer typu *decoder-only*, stanowiącą fundament modeli takich jak Llama-3.1. Omówiono proces tokenizacji, mechanizm uwagi oraz ewolucję stanów ukrytych w strumieniu residualnym. Druga część rozdziału klasyfikuje i systematyzuje dotychczasowe, literaturowe podejścia do detekcji halucynacji, pozycjonując na ich tle autorską metodę badawczą opisaną w dalszych częściach niniejszej pracy.

2.1. Architektura Transformer

Architektura Transformer, przedstawiona przez Vaswaniego i in. [7], stanowi fundament współczesnych dużych modeli językowych. W odróżnieniu od wcześniejszych podejść rekurencyjnych, Transformer przetwarza całą sekwencję wejściową równolegle, co pozwala na efektywne uchwycenie długodystansowych zależności.

Przedmiotem niniejszej pracy są modele generatywne działające w architekturze *decoder-only*. Modele tego typu, takie jak badany w eksperymentach Llama-3.1-8B, generują odpowiedź autoregresyjnie. Kolejny token x_{t+1} jest dobierany na podstawie sekwencji dotychczas wygenerowanych tokenów $x_1, \dots, x_t$. Zrozumienie wewnętrznej dynamiki tego procesu, w szczególności ewolucji stanów ukrytych i rozkładów prawdopodobieństwa w kolejnych warstwach, jest bezpośrednią podstawą metodologii detekcji halucynacji przyjętej w pracy. Rysunek 2.1 ilustruje strukturę opisywanego wariantu sieci.



Rysunek 2.1. Architektura transformera typu *decoder-only*.
Na podstawie źródła [7].

2.1.1. Tokenizacja i osadzenia

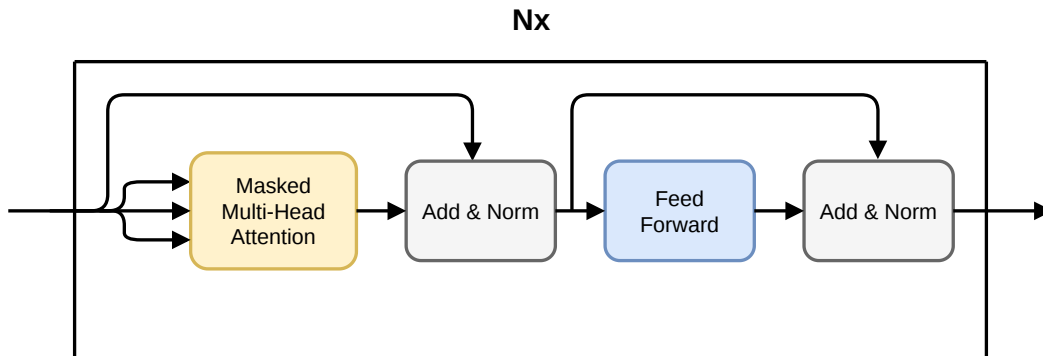
Ciąg znaków wejściowych jest przed przetwarzaniem dzielony na *tokens*, czyli skończone, zazwyczaj krótkie podciągi znaków ze skończonego słownika $\mathcal{V}$. Każdy token $x_t \in \mathcal{V}$ jest następnie mapowany na wektor osadzenia $\mathbf{e}_{x_t} \in \mathbb{R}^d$ za pomocą wyuczonej macierzy osadzeń $W_E \in \mathbb{R}^{|\mathcal{V}| \times d}$, gdzie $|\mathcal{V}|$ oznacza rozmiar słownika, a d to wymiar modelu, wspólny dla wszystkich reprezentacji wewnętrznych. Wektor $\mathbf{e}_{x_t} \in \mathbb{R}^d$ reprezentuje dany token w *przestrzeni osadzeń*.

Najważniejszą cechą wektorów osadzeń jest to, że tokeny o podobnym znaczeniu występują blisko siebie w przestrzeni osadzeń. Ponadto działania wykonywane na owej przestrzeni dobrze odwzorowują relacje między tokenami.

2.1.2. Warstwy Transformera

Warstwy Transformera stanowią kluczowy element całej architektury. W ich skład wchodzi omówiony wcześniej mechanizm maskowanej uwagi wielogłowej, dwie warstwy połączeń residualnych wraz z normalizacją (ang. *Add & Norm*) oraz blok propagacji w przód (ang. *Feed-Forward Network*).

Na potrzeby dalszych rozważań zdefiniujemy zbiór kolejnych L warstw modelu jako $\mathcal{L} = \{l_1, l_2, \dots, l_L\}$. Wizualizację struktury pojedynczego bloku przedstawiono na rysunku 2.2.



Rysunek 2.2. Blok transformera.
Wykonano na podstawie pracy [7]

2.1.3. Mechanizm uwagi

Mechanizm uwagi umożliwia każdemu tokenowi selektywne pozyskiwanie informacji z pozostałych pozycji przetwarzanej sekwencji. Niniejszy podrozdział skupia się na dokładnym wytłumaczeniu zasad działania tego mechanizmu.

Macierze zapytań, kluczy i wartości

Dla stanu ukrytego warstwy l_i i pozycji t wyznacza się trzy rzuty liniowe:

$$\mathbf{q}_t = \mathbf{h}_t^{(l_{i-1})} W^Q, \quad \mathbf{k}_t = \mathbf{h}_t^{(l_{i-1})} W^K, \quad \mathbf{v}_t = \mathbf{h}_t^{(l_{i-1})} W^V \quad (2.1)$$

gdzie $W^Q, W^K \in \mathbb{R}^{d \times d_k}$ oraz $W^V \in \mathbb{R}^{d \times d_v}$ to nauczone macierze wag rzutujące stan ukryty o wymiarze d odpowiednio na przestrzeń zapytań i kluczy o wymiarze d_k oraz na przestrzeń wartości o wymiarze d_v . W praktyce, w modelach takich jak Llama-3.1-8B, przyjmuje się $d_k = d_v = d/\mathcal{G}$, gdzie $\mathcal{G}$ to liczba głów uwagi. Zapytanie $\mathbf{q}_t$ wyraża, jakich informacji token t poszukuje, klucze $\mathbf{k}_j$ opisują dane oferowane przez token j , natomiast wartości $\mathbf{v}_j$ odpowiadają za treść przekazywaną dalej w sieci.

Macierz uwagi

Macierz uwagi pomiędzy tokenami w sekwencji wyliczana jest z poniższego wzoru:

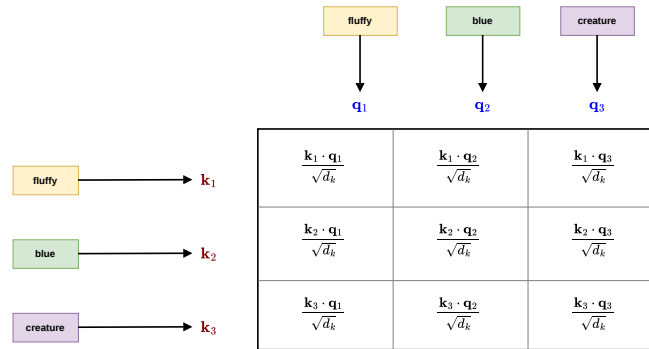
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \quad (2.2)$$

Wysoka wartość wagi uwagi świadczy o silnej istotności j -tego tokena dla i -tego tokena. Wynik podobieństwa pomiędzy tokenem i a tokenem j definiuje się jako przeskalowany iloczyn skalarny wektorów zapytania i klucza:

$$s_{ij} = \frac{1}{\sqrt{d_k}} \sum_{m=1}^{d_k} q_{im} k_{jm}, \quad (2.3)$$

gdzie czynnik $1/\sqrt{d_k}$ pełni kluczową rolę w stabilizacji wariancji obliczanego podobieństwa.

Graficzną reprezentację tego procesu przedstawiono na rysunku 2.3.



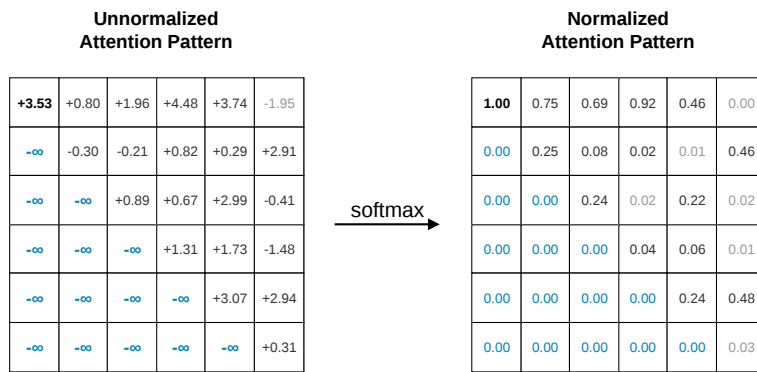
Rysunek 2.3. Macierz uwagi. Wykonano na podstawie źródła [8].

Maskowanie

W modelu dekodującym, generującym sekwencję autoregresyjnie, token na pozycji t nie może brać pod uwagę tokenów z pozycji $j > t$. Aby wymusić ten warunek, do nieznormalizowanych wyników uwagi s_{ij} przed operacją softmax dodawana jest maska:

$$M_{t,j} = \begin{cases} 0 & \text{jeśli } j \leq t, \\ -\infty & \text{jeśli } j > t, \end{cases} \quad (2.4)$$

co powoduje, że wagi uwagi dla przyszłych pozycji zerują się po operacji softmax. Schemat blokowy operacji maskowania zaprezentowano na rysunku 2.4.



Rysunek 2.4. Proces maskowania.
Wykonano na podstawie źródła [8].

Wielogłówna atencja

W modelach językowych stosuje się $\mathcal{G}$ niezależnych głów uwagi. Niech $H^{(l_{i-1})} \in \mathbb{R}^{T \times d}$ oznacza macierz stanów ukrytych wejścia do warstwy l_i . Każda głowa g dysponuje własnymi wyuczonymi macierzami rzutowania $W^{Q_g} \in \mathbb{R}^{d \times d_k}$, $W^{K_g} \in \mathbb{R}^{d \times d_k}$, $W^{V_g} \in \mathbb{R}^{d \times d_v}$:

$$Q_g = H^{l_{i-1}} W^{Q_g}, \quad K_g = H^{l_{i-1}} W^{K_g}, \quad V_g = H^{l_{i-1}} W^{V_g}, \quad (2.5)$$

$$\text{head}_g = \text{Attention}(Q_g, K_g, V_g). \quad (2.6)$$

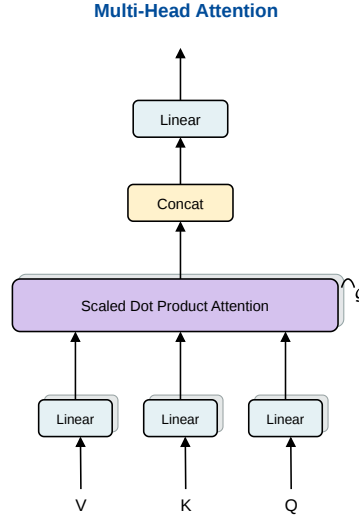
Wyniki wszystkich głów są konkatelowane i rzutowane z powrotem na przestrzeń modelu (rysunek 2.5):

$$\text{MultiHead}(H^{l_{i-1}}) = [\text{head}_1, \dots, \text{head}_{\mathcal{G}}] W_O, \quad (2.7)$$

gdzie $W_O \in \mathbb{R}^{d_v \times d}$ to macierz projekcji wyjścia. Każda głowa wyspecjalizowuje się w wychwytywaniu odmiennych wzorców zależności pomiędzy tokenami.

2.1.4. Warstwa propagacji w przód

Każda warstwa Transformer zawiera, obok komponentu uwagi, dwuwarstwową sieć w pełni połączoną (ang. *feed-forward network*, FFN), stosowaną niezależnie do reprezentacji każdego



Rysunek 2.5. Wielogłówna atencja, wykonano na podstawie źródła [7].

tokenu. Dla stanu wejściowego $\mathbf{x} \in \mathbb{R}^d$ klasyczna postać bloku FFN jest zdefiniowana następująco:

$$\text{FFN}(\mathbf{x}) = \text{Act}(\mathbf{x}W_1 + \mathbf{b}_1)W_2 + \mathbf{b}_2, \quad (2.8)$$

gdzie Act oznacza funkcję aktywacji, $W_1 \in \mathbb{R}^{d \times d_{\text{ff}}}$ jest macierzą rzutującą reprezentację wejściową do przestrzeni ukrytej o wymiarze d_{ff} , natomiast $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d}$ rzutuje wynik z powrotem do przestrzeni modelu o wymiarze d . Wektory $\mathbf{b}_1 \in \mathbb{R}^{d_{\text{ff}}}$ oraz $\mathbf{b}_2 \in \mathbb{R}^d$ stanowią wyuczone przesunięcia. Zazwyczaj wymiar warstwy ukrytej jest kilkukrotnie większy od wymiaru modelu. W oryginalnej architekturze Transformer przyjęto $d_{\text{ff}} = 4d$.

Komponent FFN jest często interpretowany jako struktura pamięciowa modelu. Przechowuje wzorce i zależności nabyte podczas treningu, które mogą być następnie wykorzystywane przez mechanizm atencji. W literaturze wskazuje się, że nieprawidłowości w tej pamięci stanowią jedną z istotnych przyczyn występowania halucynacji modeli językowych.

2.1.5. Normalizacja warstw

Normalizacja warstw stabilizuje proces uczenia przez sprowadzenie aktywacji do rozkładu o zerowej średniej i wariancji równej jeden:

$$\text{LN}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu(\mathbf{x})}{\sigma(\mathbf{x}) + \varepsilon} + \beta, \quad (2.9)$$

gdzie $\mu(\mathbf{x}) = \frac{1}{d} \sum_{i=1}^d x_i$ to średnia współrzędnych wektora $\mathbf{x}$, $\sigma(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d (x_i - \mu(\mathbf{x}))^2}$ to ich odchylenie standardowe, $\varepsilon > 0$ to stała zapewniająca stabilność numeryczną, a $\gamma, \beta \in \mathbb{R}^d$ to wyuczone parametry przeskalowania i przesunięcia.

2.1.6. Strumień residualny

Kluczową strukturą danych w Transformerze, istotną dla analizy stanów wewnętrznych modelu, jest *strumień residualny*. Dla każdego tokenu t strumień residualny jest wektorem $\mathbf{h}_t \in \mathbb{R}^d$, który na początku zawiera jedynie osadzenie tokenu i aktualizowany jest addytywnie przez każdą warstwę modelu. Równanie przedstawiające opisywany przepływ informacji pokazane jest poniżej:

$$\mathbf{h}_t^{(l_i)} = \mathbf{h}_t^{(l_{i-1})} + \Delta_{\text{Attn}}^{(l_i)}(\mathbf{h}^{(l_{i-1})}) + \Delta_{\text{FFN}}^{(l_i)}(\mathbf{h}^{(l_{i-1})}), \quad (2.10)$$

gdzie $\Delta_{\text{Attn}}^{(l_i)}$ i $\Delta_{\text{FFN}}^{(l_i)}$ oznaczają odpowiednio wkłady warstwy uwagi i warstwy propagacji w przód.

Należy zauważyć, że informacja z warstw wejściowych nie zanika, lecz jest akumulowana, ponieważ każdy stan $\mathbf{h}_t^{(l_i)}$ zawiera zarówno informację z warstwy wejściowej, jak i informację przetworzoną przez warstwy $l_1, \dots, l_i$.

2.1.7. Stany ukryte

Stan ukryty tokenu t po warstwie l_i to wektor

$$\mathbf{h}_t^{(l_i)} \in \mathbb{R}^d, \quad (2.11)$$

będący aktualną zawartością strumienia residualnego, gdzie d to wymiar modelu.

Stany ukryte są nośnikami informacji kontekstowej akumulowanej w miarę przetwarzania sekwencji przez kolejne warstwy. Na warstwach początkowych reprezentacje odzwierciedlają przede wszystkim właściwości leksykalne tokenu, natomiast w warstwach głębszych kodują coraz bardziej abstrakcyjne zależności semantyczne i relacje w tekście [9].

Odległość kosinusową między wektorami $\mathbf{h}_t^{(l_i)}$ i $\mathbf{h}_s^{(l_i)}$ traktujemy w niniejszej pracy jako roboczą miarę zmiany reprezentacji kontekstowej modelu. Nie zakładamy, że jest ona pełną miarą podobieństwa semantycznego w sensie lingwistycznym. Używamy jej jako sygnału geometrycznego, który może korelować ze zmianą sposobu przetwarzania tokenu przez model. Analiza trajektorii stanów ukrytych przez kolejne warstwy może ujawniać nieprawidłowości, takie jak nagłe zmiany kierunku reprezentacji, które stanowią jeden z sygnałów badanych w niniejszej pracy.

Nowsze modele, w tym Llama-3.1-8B, stosują *pre-normalizację*, polegającą na zastosowaniu operacji normalizacji przed każdym blokiem uwagi i FFN. Jedna iteracja aktualizacji strumienia residualnego przyjmuje postać:

$$\mathbf{a}_t^{(l_i)} = \mathbf{h}_t^{(l_{i-1})} + \text{Attention}^{(l_i)}(\text{LN}_1^{(l_i)}(\mathbf{h}^{(l_{i-1})})), \quad (2.12)$$

$$\mathbf{h}_t^{(l_i)} = \mathbf{a}_t^{(l_i)} + \text{FFN}^{(l_i)}(\text{LN}_2^{(l_i)}(\mathbf{a}_t^{(l_i)})), \quad (2.13)$$

gdzie $\mathbf{a}_t^{(l_i)} \in \mathbb{R}^d$ to stan pośredni po warstwie uwagi, a $\text{LN}_1^{(l_i)}$, $\text{LN}_2^{(l_i)}$ to niezależne operacje normalizacji warstwowej (sekcja 2.1.5).

Powyższy opis przedstawia uproszczony blok transformera. Model używany w eksperymentach, Llama-3.1-8B [1], stosuje zaawansowane warianty tej architektury. Wykorzystywana

jest normalizacja RMSNorm zamiast LayerNorm oraz bramkowane warstwy MLP z aktywacją SwiGLU zamiast prostej sieci FFN z aktywacją GELU.

2.1.8. Dekodowanie

Po przejściu przez L warstw transformera finalna reprezentacja $\mathbf{h}_t^{(l_L)} \in \mathbb{R}^d$ poddawana jest transformacji liniowej przez *macierz odosadzania* (ang. *unembedding*) $W_U \in \mathbb{R}^{|\mathcal{V}| \times d}$, która rzutuje wektor z przestrzeni modelu o wymiarze d na przestrzeń słownika o wymiarze $|\mathcal{V}|$. Każdy wiersz $W_U[k, :] \in \mathbb{R}^d$ to wektor reprezentujący token k w przestrzeni ukrytej, przy czym logit tokenu k jest iloczynem skalarnym tego wektora ze stanem $\mathbf{h}_t^{(l_L)}$, więc token uzyskuje wysokie prawdopodobieństwo, gdy stan ukryty jest zbliżony kierunkowo do $W_U[k, :]$.

$$\mathbf{z}_t^{(l_L)} = \mathbf{h}_t^{(l_L)} W_U^\top \in \mathbb{R}^{|\mathcal{V}|}, \quad (2.14)$$

gdzie $\mathbf{z}_t^{(l_L)}$ to wektor logitów dla pozycji t , a każdy logit $z_{t,k}^{(l_L)}$ wyraża preferencję modelu dla tokenu k jako następnego tokenu w sekwencji. Operacja softmax przekształca logity w prawdopodobieństwa:

$$p_{t,k}^{(l_L)} = \frac{\exp(z_{t,k}^{(l_L)})}{\sum_{j=1}^{|\mathcal{V}|} \exp(z_{t,j}^{(l_L)})}, \quad k \in \{1, \dots, |\mathcal{V}|\}, \quad (2.15)$$

przy czym $p_{t,k}^{(l_L)}$ interpretuje się jako prawdopodobieństwo modelu przypisane tokenowi k na pozycji $t + 1$.

2.1.9. Model Llama-3.1-8B

W przeprowadzonych eksperymentach wykorzystano model Llama-3.1-8B [1] typu decoder-only, zawierający około 8 miliardów parametrów. Model składa się z 32 warstw transformera, a wymiar reprezentacji ukrytej wynosi $d = 4096$. Oznacza to, że stan ukryty każdego tokenu w każdej warstwie jest reprezentowany przez wektor należący do przestrzeni $\mathbb{R}^{4096}$. Słownik tego modelu zawiera 128 256 tokenów.

2.2. Charakterystyka i detekcja halucynacji

2.2.1. Definicja

Halucynacje (ang. *hallucinations*) to zjawisko, w którym model językowy generuje tekst brzmiący przekonująco i płynnie, lecz niezgodny z faktami, z dostarczonym kontekstem lub z własną wcześniejszą odpowiedzią [10, 11]. Termin ten jest używany szeroko, lecz różne prace przyjmują odmienne definicje. W niniejszym rozdziale przyjmujemy konwencję zbliżoną do przeglądów Ji i in. [10].

Cytowany wyżej tekst wymienia dwa główne rodzaje halucynacji. Halucynacje wewnętrzne (ang. *intrinsic*) zachodzą, gdy odpowiedź jest sprzeczna z dostarczonym kontekstem lub

dokumentami. Halucynacje zewnętrzne (ang. *extrinsic*) polegają na tym, że odpowiedź nie wynika ani z kontekstu, ani z weryfikowalnej wiedzy. Model generuje nieistniejące cytaty, daty lub fakty, których nie da się łatwo zweryfikować ani obalić na podstawie dostarczonego materiału.

2.2.2. Pseudohalucynacje

W literaturze badającej zjawisko halucynacji etykietowanie danych do treningu i ewaluacji detektorów niemal zawsze sprowadza się do oceny niezgodności wygenerowanej odpowiedzi z dostępnymi tekstami referencyjnymi przy użyciu różnych metod (do oceny wykorzystywane są inne modelu lub algorytmy), nie zaś do bezpośredniej weryfikacji jej prawdziwości. Dotyczy to również niemal wszystkich metod omówionych w niniejszym rozdziale. Podejście takie wiąże się z nieuniknionym szumem etykiet, ponieważ odpowiedź merytorycznie poprawna, lecz odbiegająca od referencji, może zostać błędnie oznaczona jako halucynacja. Niniejsza praca również przyjmuje to podejście. Zatem wykrywane zjawisko określane jest w niniejszej pracy mianem *pseudohalucynacji*, jako odpowiedzi niezgodnej z tekstem referencyjnym.

2.2.3. Rodziny metod detekcji halucynacji

Metody detekcji halucynacji można podzielić według wymaganego dostępu do modelu i źródła sygnału detekcyjnego. Poniżej omówiono pięć głównych rodzin.

Metody czarnoskrzynkowe oparte na spójności odpowiedzi

Metody czarnoskrzynkowe (ang. *black-box*) operują wyłącznie na wejściu i wyjściu modelu, nie wymagając dostępu do stanów wewnętrznych ani wag.

Reprezentatywną metodą tej klasy jest SelfCheckGPT [12]. Metoda ocenia spójność wielu odpowiedzi generowanych na to samo pytanie, zakładając, że przy halucynacji odpowiedzi są mniej zgodne ze sobą. SelfCheckGPT nie wymaga dostępu do parametrów modelu. Wadą metody jest koszt obliczeniowy, ponieważ detekcja wymaga wygenerowania kilku do kilkunastu odpowiedzi na jedno pytanie.

Pokrewną metodą jest entropia semantyczna (ang. *semantic entropy*) [13]. Metoda mierzy niepewność semantyczną na podstawie wielu odpowiedzi generowanych na to samo pytanie. W przeciwieństwie do SelfCheckGPT, metoda ta ocenia spójność na poziomie znaczeniowym, dzięki czemu synonimy, parafrazy odpowiedzi nie są traktowane jako rozbieżne.

Metody białoskrzynkowe oparte na stanach ukrytych

Metody oparte na stanach ukrytych zakładają, że sieć neuronowa pozostawia ślad w przestrzeni reprezentacji wewnętrznej, kiedy generuje treść nieprawdziwą.

Pionierskie badania w tym obszarze przeprowadzili Azaria i Mitchell [14]. Wykazali oni, że stany ukryte generowane przez model podczas wypowiadania stwierdzeń prawdziwych i fałszywych wykazują odmienną charakterystykę geometryczną. Jako sygnał wejściowy autorzy wykorzystali wektory aktywacji (stany ukryte) ekstrahowane z warstw środkowych i końcowych modelu. Na tych reprezentacjach wytrenowali zewnętrzny klasyfikator (prostą sieć wielowarstwową MLP), który był w stanie przewidzieć prawdziwość generowanego zdania.

Badanie to dowiodło, że informacja o faktograficznej poprawności nie jest rozproszona losowo, lecz tworzy klastry w przestrzeni modelu, a najsilniejszy sygnał korelujący z halucynacjami występuje w głębszych warstwach sieci.

Z kolei Su i in. [15] rozwinęli tę koncepcję, eliminując konieczność trenowania zewnętrznego klasyfikatora nadzorowanego, co stanowi bezpośredni punkt odniesienia dla niniejszej pracy. Autorzy zaproponowali podejście w pełni nienadzorowane, bazujące wyłącznie na analizie trajektorii i geometrii stanów ukrytych z pojedynczego przebiegu generowania tekstu. Kluczowym sygnałem skorelowanym z halucynacjami okazała się w ich pracy dynamiczna zmienność kierunku wektorów aktywacji warstwach sieci. Zaobserwowano, że podczas generowania poprawnych odpowiedzi trajektorie stanów ukrytych w strumieniu residualnym są stabilne i zbiegają ku konkretnym reprezentancjom semantycznym. W przypadku halucynacji dochodzi natomiast do nagłych perturbacji geometrycznych pomiędzy warstwami, co sygnalizuje brak pewności modelu lub konflikt wewnętrzny pomiędzy pamięcią FFN a mechanizmem uwagi.

Hu i in. [16] zaproponowali metodę HARP (*Hallucination Detection via Reasoning Subspace Projection*), która wykrywa halucynacje poprzez rzutowanie wewnętrznych aktywacji modelu na podprzestrzeń rozumowania. Zakłada się, że poprawne odpowiedzi i pseudohalucynacje pozostawiają odmienną sygnaturę w podprzestrzeniach przestrzeni ukrytej. Autorzy pracy wykorzystują surowe wektory ukryte po operacji rzutowania, nie wykorzystują interpretowalnych cech.

Metody oparte na analizie macierzy uwagi

Osobną klasę stanowią podejścia, które wykorzystują macierze uwagi do identyfikacji błędów w generowanym tekście. Prace LapEigvals [17] oraz TOHA [18] opierają się na założeniu, że struktura połączeń między tokenami w macierzach uwagi pozwala skutecznie wykrywać halucynacje modelu.

Metody oparte na miarach niepewności probabilistycznej

Osobną grupę rozwiązań stanowią podejścia opierające się na niepewności predykcyjnej modelu, traktujące stopień wahania sieci jako główny wyznacznik podatności na halucynowanie.

W tej grupie istotną rolę odgrywa analiza zakłopotania (ang. *perplexity*), rozumianego jako geometryczna średnia odwrotności prawdopodobieństw przypisanych generowanym tokenom. Kluczowe znaczenie ma tutaj praca Ren i in. [19], w której autorzy badają *perplexity* w kontekście modeli językowych.

Najnowsze trendy koncentrują się na agregacji sygnałów niepewności wzdłuż całej osi czasu generowania tekstu. W publikacji Sharmy i in. [20], zatytułowanej *Think Just Enough*, wprowadzono koncepcję entropii na poziomie całej sekwencji (ang. *sequence-level token entropy*) jako bezpośredniego wyznacznika pewności sieci.

Metody oparte na zewnętrznych źródłach wiedzy

Metody zewnętrzne (ang. *external evidence*) weryfikują odpowiedź modelu względem dokumentów lub zewnętrznej bazy wiedzy. FActScore [21] weryfikuje odpowiedź modelu poprzez porównanie z zewnętrzną bazą wiedzy.

Z kolei systemy RAGTruth [22] i Ragas [23] oceniają wierność odpowiedzi względem dostarczonego kontekstu, co jest szczególnie istotne w systemach generowania wspomaganego wyszukiwaniem (RAG). Metody zewnętrzne zapewniają najsilniejszy dowód poprawności, ale wymagają dostępu do bazy danych lub systemu wyszukiwania w trakcie detekcji.

2.2.4. Pozycjonowanie niniejszej pracy

Niniejsza praca przedstawia metodę białoskrzynkową, która zakłada dostęp do reprezentacji wewnętrznych oraz wyjściowych logitów modelu, rezygnując jednocześnie z konieczności generowania wielu wariantów tekstu czy odpytywania zewnętrznych baz wiedzy. Podsumowanie najważniejszych właściwości omawianych metod zostało zestawione w tabeli 2.1.

Metoda	Główna rodzina	Źródło sygnału	Gen. wielu odpow.	Zewn. dane
SelfCheckGPT [12]	czarnoskrzynkowa	tekst	tak	nie
Entropia semantyczna [13]	czarnoskrzynkowa	tekst	tak	nie
FActScore [21]	czarnoskrzynkowa	tekst	nie	baza wiedzy
RAGTruth [22] / Ragas [23]	czarnoskrzynkowa	tekst	nie	dok. RAG
Ren i in. [19]	białoskrzynkowa	logity	nie	nie
Think Just Enough [20]	białoskrzynkowa	logity	nie	nie
Azaria i Mitchell [14]	białoskrzynkowa	stany ukryte	nie	nie
Su i in. [15]	białoskrzynkowa	stany ukryte	nie	nie
HaloScope [24]	białoskrzynkowa	stany ukryte	nie	nie
HARP [16]	białoskrzynkowa	stany ukryte	nie	nie
LapEigvals [17]	białoskrzynkowa	macierze uwagi	nie	nie
TOHA [18]	białoskrzynkowa	macierze uwagi	nie	dok. RAG
Niniejsza praca	białoskrzynkowa	stany ukryte + logity	nie	nie

Tabela 2.1. Porównanie metod detekcji halucynacji. Kolumna *Zewn. dane* wskazuje wymagany dostęp do bazy wiedzy lub dokumentów referencyjnych.

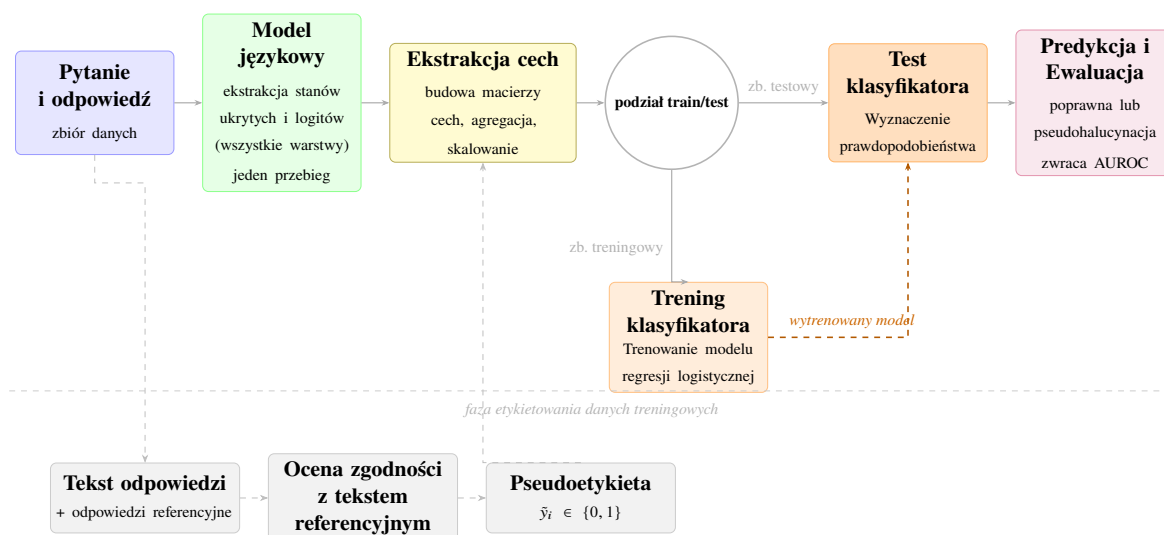
W odróżnieniu od klasycznych technik czarnoskrzynkowych, takich jak SelfCheckGPT czy entropia semantyczna, proponowane podejście nie generuje narzutu obliczeniowego związanego z wielokrotnym próbkowaniem odpowiedzi. Z kolei na tle innych metod białoskrzynkowych naszą pracę wyróżnia specyfika wykorzystywanego sygnału. W przeciwieństwie do algorytmów LapEigvals oraz TOHA rezygnujemy z analizy map uwagi, a w porównaniu do podejścia HARP [16], które operuje na surowych aktywacjach rzutowanych przez wyuczone podprzestrzenie, celowo wybrano badanie interpretowalnych cech ekstrahowanych ze stanów ukrytych modelu.

Rozdział 3

Metodologia badań

3.1. Potok badawczy

Niniejsza sekcja wprowadza czytelnika w ogólny schemat potoku badawczego, zilustrowany na rysunku 3.1. Dla każdego pytania ze zbiorów TriviaQA, SQuAD i NQOpen model językowy generuje odpowiedź, która następnie rozgałęzia się na dwie ścieżki. Pierwsza z nich służy etykietowaniu, tj. automatycznej ocenie zgodności wygenerowanej odpowiedzi z tekstem referencyjnym i przypisaniu pseudoetykiety $\tilde{y}_i \in \{0, 1\}$. Druga ścieżka obejmuje ekstrakcję cech wewnętrznych modelu, tj. budowę macierzy Φ_i ze stanów ukrytych i logitów zbieranych ze wszystkich badanych warstw. Tak przygotowane dane przekazywane są do klasyfikatora, który na zbiorze treningowym uczy się odróżniać odpowiedzi poprawne od pseudohalucynacji, a na zbiorze testowym wyznaczane jest AUROC. Szczegółowy opis każdego z etapów znajduje się w późniejszej części niniejszej pracy.



Rysunek 3.1. Schemat potoku białoskrzynkowej detekcji pseudohalucynacji.

3.2. Pozyskiwanie i przetwarzanie danych

3.2.1. Generowanie odpowiedzi

Wszystkie odpowiedzi na pytania wykorzystane w eksperymentach zostały wygenerowane za pomocą modelu meta-llama/Llama-3.1-8B i biblioteki transformers w wersji 5.10.2. W procesie generacji wykorzystano parametry `max_new_tokens=200`, `do_sample=True` oraz temperatura $\tau = 0,7$. Dodatkowo ustawiono stałe ziarno losowości o wartości 42. Algorytm 1 przedstawia procedurę generowania odpowiedzi. Procesy trenowania i testowania klasyfikatorów realizowano wyłącznie na danych pochodzących z tego modelu. Model

generował odpowiedzi na pytania pochodzące z trzech repozytoriów dostępnych publicznie na platformie Hugging Face: TriviaQA, NQOpen oraz SQuAD. Dobór tych zbiorów podyktowany był potrzebą analizy obu rodzajów pseudohalucynacji wymienionych w sekcji 2.2.2. Zbiory TriviaQA oraz NQOpen posłużyły do badania pseudohalucynacji zewnętrznych, ponieważ wymagają one od modelu odwołania się do wiedzy ogólnej bez dostępu do zewnętrznego tekstu źródłowego. Z kolei zbiór SQuAD wykorzystano do analizy pseudohalucynacji wewnętrznych, gdyż zadanie polega w tym przypadku na ekstrakcji wiedzy z dostarczonego kontekstu, co umożliwia bezpośrednią weryfikację spójności wygenerowanej odpowiedzi ze źródłem. Dla każdego pytania wygenerowano pojedynczą odpowiedź. Wygenerowane odpowiedzi i referencje są zapisywane, a po etykietowaniu dopisywana jest pseudoetykieta. Etykietowanie jest kosztowne obliczeniowo, ponieważ każda odpowiedź porównywana jest z wieloma tekstami referencyjnymi. Zapis wyników tego kroku pozwala wielokrotnie uruchamiać eksperymenty bez konieczności powtarzania etykietowania. Dokładny opis zbiorów danych i struktury promptów przedstawiony jest w dodatku A.1. Ze względu na ograniczenia zasobów obliczeniowych, z każdego wymienionego zbioru wybrano próbę początkowych 10 000 pytań. Dane te zostały następnie podzielone na zbiór treningowy i testowy w proporcji 80:20.

Algorytm 1 Generowanie i zapis odpowiedzi

Require: zbiór pytań $\{q_i\}$ wraz z referencjami $\{A_i^*\}$, model $\mathcal{M}$, tokenizer $\mathcal{T}$, temperatura $\tau = 0,7$, limit tokenów $N_{\max} = 200$

Ensure: zbiór rekordów $\{(q_i, \hat{a}_i, A_i^*)\}$

- 1: **for** każde pytanie q_i **do**
 - 2: $\mathbf{x}_i \leftarrow \mathcal{T}.\text{TOKENIZE}(q_i)$
 - 3: $\hat{a}_i \leftarrow \mathcal{M}.\text{GENERATE}(\mathbf{x}_i, \tau = 0,7, \text{do_sample} = \text{True}, \text{max_new_tokens} = N_{\max})$
 - 4: Zapisz rekord $(q_i, \hat{a}_i, A_i^*)$
 - 5: **end for**
-

3.2.2. Etykietowanie

Ocena poprawności wygenerowanej odpowiedzi opiera się na automatycznym porównaniu z tekstem referencyjnym przy użyciu dwóch metod, którymi były *BLEURT* [5] oraz *ROUGE-L* [6]. W eksperymentach użyto checkpointu BLEURT-20, który jest najnowszą publicznie dostępną wersją *BLEURT*. Niech $A^* = \{a_1^*, \dots, a_m^*\}$ oznacza zbiór dostępnych odpowiedzi referencyjnych dla danego pytania. Zdefiniowano wyniki porównania wygenerowanej odpowiedzi $\hat{a}$ z tym zbiorem:

$$b_{\text{BL}}(\hat{a}, A^*) = \max_{a \in A^*} \text{BLEURT}(\hat{a}, a), \quad (3.1)$$

$$b_{\text{RL}}(\hat{a}, A^*) = \max_{a \in A^*} \text{ROUGE-L}(\hat{a}, a). \quad (3.2)$$

Pseudoetykieta $\tilde{y}_i \in \{0, 1\}$ przypisana odpowiedzi $\hat{a}$ przyjmuje wartość:

$$\tilde{y}_i = \begin{cases} 0 & \text{jeśli } b_{\text{BL}}(\hat{a}, A^*) > 0,5 \text{ lub } b_{\text{RL}}(\hat{a}, A^*) > 0,7, \\ 1 & \text{w przeciwnym razie.} \end{cases} \quad (3.3)$$

Wartość $\tilde{y}_i = 1$ oznacza niską zgodność z tekstem referencyjnym i jest stosowana w pracy jako przybliżenie etykiety pseudohalucynacji określanej w tej pracy mianem pseudohalucynacji. ROUGE-L mierzy podobieństwo sekwencji na podstawie najdłuższego wspólnego podciągu (ang. *Longest Common Subsequence*), natomiast BLEURT jest wyuczoną metryką oceny podobieństwa semantycznego opartą na modelu BERT. Zaprezentowane podejście oraz wartości progowe (0,5 dla metryki BLEURT oraz 0,7 dla ROUGE-L) zostały przyjęte na podstawie pracy Hu i in. [16]. Progi te zastosowano w pierwotnej formie, bez żadnych modyfikacji.

3.2.3. Ograniczenia etykietowania

Zastosowane podejście do etykietowania niesie ze sobą kilka istotnych ograniczeń, które należy uwzględnić przy interpretacji wyników. Pseudoetykieta $\tilde{y}_i = 1$ sygnalizuje niską zgodność leksykalną lub semantyczną z dostępnymi odpowiedziami referencyjnymi, lecz nie stanowi pewnego dowodu błędu merytorycznego. Model mógł wygenerować odpowiedź poprawną, lecz różniącą się sformułowaniem od wzorca (tzw. *false positive* etykietowania). Zarówno BLEURT, jak i ROUGE-L mierzą powierzchowne podobieństwo tekstowe, nie weryfikując prawdziwości twierdzeń.

3.2.4. Ekstrakcja macierzy cech

Po wygenerowaniu odpowiedzi wykonywany jest dodatkowy przebieg modelu (*forward pass*) dla całej sekwencji wejściowej, obejmującej zarówno prompt, jak i wygenerowaną odpowiedź. W trakcie tego przebiegu zapisywane są stany ukryte wszystkich analizowanych warstw oraz odpowiadające im logity. Na podstawie uzyskanych danych budowana jest macierz cech $\Phi_i \in \mathbb{R}^{T_a \times D}$ dla każdej wygenerowanej odpowiedzi. Rozmiar D jest stały i wynosi $D = |\mathcal{L}| \cdot N_{\text{cech}}$, gdzie $|\mathcal{L}|$ to liczba badanych warstw, a N_{cech} to liczba cech wyliczanych z pojedynczej warstwy. Opis wszystkich badanych cech zawiera sekcja 3.3, natomiast algorytm 2 przedstawia ekstrakcję macierzy cech.

Pełna macierz cech dla jednej wygenerowanej odpowiedzi i reprezentowana jest jako:

$$\Phi_i \in \mathbb{R}^{T_a \times (|\mathcal{L}| \cdot N_{\text{cech}})}. \quad (3.4)$$

3.2.5. Zagregowany wektor cech

Pełna macierz cech $\Phi_i \in \mathbb{R}^{T_a \times D}$, zdefiniowana w równaniu (3.4), posiada zmienną liczbę wierszy T_a , zależną od długości generowanej odpowiedzi. Klasyfikator oparty na regresji logistycznej wymaga jednak wektora wejściowego o stałym wymiarze. W tym celu dla każdej kolumny $j \in \{1, \dots, D\}$ obliczane są trzy statystyki agregujące: średnia, odchylenie

Algorytm 2 Ekstrakcja macierzy cech dla jednej odpowiedzi

Require: stany ukryte H_i , logity Z_i , zbiór warstw $\mathcal{L}$

Ensure: macierz cech $\Phi_i \in \mathbb{R}^{T_a \times D}$

```

1: Wyodrębnij indeksy tokenów należących do odpowiedzi:  $t \in \mathcal{A} = \{q, \dots, T-1\}$ 
2: for każdy token  $t \in \mathcal{A}$  do
3:   for każdą warstwę  $l_i \in \mathcal{L}$  do
4:     Oblicz wektor cech warstwy:  $\mathbf{x}_t^{(l_i)} \in \mathbb{R}^{N_{\text{cech}}}$  ze stanów  $\mathbf{h}_t^{(l_i)}$  i logitów  $\tilde{\mathbf{z}}_t^{(l_i)}$ 
5:   end for
6:   Złącz wektory ze wszystkich warstw w jeden wektor cech tokenu  $t$ :
7:    $\Phi_{i,t} = [\mathbf{x}_t^{(l_1)}; \mathbf{x}_t^{(l_2)}; \dots, \mathbf{x}_t^{(l_L)}] \in \mathbb{R}^D$ 
8: end for
9: Zbuduj macierz  $\Phi_i$ , układając wektory  $\Phi_{i,t}$  jako kolejne wiersze:
10:  $\Phi_i \leftarrow \begin{bmatrix} \Phi_{i,q} \\ \vdots \\ \Phi_{i,T-1} \end{bmatrix} \in \mathbb{R}^{T_a \times D}$ 
11: return  $\Phi_i$ 
  
```

standardowe oraz maksimum.

$$\bar{\phi}_{i,j} = \frac{1}{|\mathcal{A}|} \sum_{t \in \mathcal{A}} (\Phi_i)_{t,j}, \quad (3.5)$$

$$\sigma_{i,j} = \sqrt{\frac{1}{|\mathcal{A}|} \sum_{t \in \mathcal{A}} ((\Phi_i)_{t,j} - \bar{\phi}_{i,j})^2}, \quad (3.6)$$

$$e_{i,j} = \max_{t \in \mathcal{A}} (\Phi_i)_{t,j}. \quad (3.7)$$

Wynikowy wektor zagregowanych cech dla odpowiedzi i ma postać:

$$\mathbf{f}_i = (\bar{\phi}_{i,1}, \sigma_{i,1}, e_{i,1}, \dots, \bar{\phi}_{i,D}, \sigma_{i,D}, e_{i,D}) \in \mathbb{R}^{3D}. \quad (3.8)$$

Wymiar wynikowego wektora wynosi $3D = 3 \cdot |\mathcal{L}| \cdot N_{\text{cech}}$, gdzie $N_{\text{cech}} = 8$ oznacza liczbę analizowanych cech (sekcja 3.3). Tak zdefiniowany wektor $\mathbf{f}_i$ stanowi reprezentację pojedynczej odpowiedzi wykorzystywaną przez klasyfikator do detekcji pseudohalucynacji.

3.2.6. Skalowanie cech

We wszystkich etapach eksperymentów zastosowano metodę *RobustScaler* z biblioteki Scikit-learn [25]. Wartość pojedynczej cechy x jest przesuwana względem mediany i skalowana przez rozstęp międzykwartyłowy (IQR), wyznaczone **wyłącznie** na podstawie próby treningowej:

$$\tilde{x} = \frac{x - \text{med}(x_{\text{train}})}{\text{IQR}(x_{\text{train}})}, \quad (3.9)$$

gdzie $\text{IQR}(x_{\text{train}}) = Q_{0,75}(x_{\text{train}}) - Q_{0,25}(x_{\text{train}})$. Wybór metody podyktowany jest odpornością na wartości odstające. Mediana i IQR są statystykami odpornymi na wartości odstające, wobec czego ekstrema nie zniekształcają transformacji.

Skalowanie zagregowanego wektora cech. Po operacji agregacji dla odpowiedzi i (sekcja 3.2.5) otrzymuje się wektor $\mathbf{f}_i \in \mathbb{R}^{3D}$. Przed przekazaniem go do modelu regresji logistycznej, każdy element $k \in \{1, \dots, 3D\}$ tego wektora jest indywidualnie skalowany zgodnie z zależnością (3.9), z wykorzystaniem parametrów dopasowanych do zbioru treningowego. Zabieg ten zapobiega dominacji cech o dużych wartościach bezwzględnych nad zmiennymi o mniejszym zakresie, a także stabilizuje proces optymalizacji wag regularyzowanych L_1 .

3.3. Badane cechy

W niniejszym rozdziale opisano cechy ekstrahowane z modelu w trakcie przetwarzania odpowiedzi. Służą one jako predyktory w procesie klasyfikacji sekwencji jako pseudohalucynacja. Sygnały te są pozyskiwane bezpośrednio z logitów, wyjściowych rozkładów prawdopodobieństwa tokenów oraz stanów ukrytych ekstrahowanych na przestrzeni kolejnych warstw modelu. Dla każdej cechy sformułowano hipotezę dotyczącą jej związku z występowaniem pseudohalucynacji. Sformułowane hipotezy są weryfikowane w rozdziale 4. Tabela 3.1 zestawia symbole stosowane w całym rozdziale.

Symbol	Znaczenie
x_t	token na pozycji t
T_p	liczba tokenów promptu
T_a	liczba tokenów odpowiedzi
$T = T_p + T_a$	całkowita długość sekwencji
$\mathcal{A}$	zbiór indeksów tokenów odpowiedzi
$\mathbf{h}_t^{(l_i)}$	stan ukryty pozycji t po warstwie l_i , $\in \mathbb{R}^d$
L	liczba warstw transformera
$\mathcal{L}$	wybrane warstwy do ekstrakcji cech
W_U	macierz odosadzania (ang. <i>unembedding matrix</i>), $\in \mathbb{R}^{ \mathcal{V} \times d}$
$\tilde{\mathbf{z}}_t^{(l_i)}$	pełny wektor logitów dla tokenu x_t z prefiksu kończącego się na $t-1$
$\mathcal{K}_t$	zbiór top- K kandydatów wybrany z finalnej warstwy
$p_{t,k}^{(l_i)}$	prawdopodobieństwo kandydata $k \in \mathcal{K}_t$ w warstwie l_i
Φ	macierz cech jednej odpowiedzi, $\in \mathbb{R}^{T_a \times D}$
$\tilde{y}_i$	pseudoetykieta odpowiedzi i ($0 =$ poprawna, $1 =$ pseudohalucynacja)

Tabela 3.1. Zestawienie symboli używanych w sekcji 3.3.

3.3.1. Formalizmy

Niech L oznacza liczbę warstw transformera. Dla sekwencji wejściowej

$$\mathbf{x} = (x_1, \dots, x_T) \in \mathcal{V}^T,$$

gdzie $\mathcal{V}$ jest słownikiem modelu, przez

$$\mathbf{h}_t^{(l_i)} \in \mathbb{R}^d$$

oznaczamy stan ukryty (ang. *hidden state*) tokenu na pozycji t po przejściu przez warstwę l_i , gdzie d jest wymiarem przestrzeni reprezentacji modelu. Warstwę osadzeń (ang. *embedding layer*) oznaczamy przez l_0 , natomiast l_L odpowiada wyjściu ostatniej warstwy transformera.

Sekwencja wejściowa x stanowi konkatenację promptu p oraz wygenerowanej odpowiedzi a , co zapisujemy jako

$$x = p \parallel a.$$

Jej długość wynosi $T = T_p + T_a$, gdzie T_p i T_a oznaczają odpowiednio liczbę tokenów promptu i odpowiedzi.

Przyjmujemy indeksowanie od zera. Tokeny promptu zajmują pozycje

$$0, \dots, T_p - 1,$$

natomiast tokeny odpowiedzi pozycje

$$q, q + 1, \dots, T - 1,$$

gdzie $q = T_p$ oznacza indeks pierwszego tokenu odpowiedzi. Wszystkie analizowane w dalszej części pracy cechy obliczane są wyłącznie dla tokenów odpowiedzi, tj. dla pozycji należących do zbioru

$$\mathcal{A} = \{q, q + 1, \dots, T - 1\}.$$

Zbiór analizowanych warstw definiujemy jako

$$\mathcal{L} = \{l_1, l_2, \dots, l_L\}.$$

Warstwa osadzeń l_0 nie należy do zbioru $\mathcal{L}$ i nie jest uwzględniana w dalszej analizie. W eksperymentach uwzględniono wszystkie warstwy transformera, tzn. $|\mathcal{L}| = L$.

3.3.2. Definicje pomocnicze

Definicja 3.1. Odległość cosinusowa między wektorami $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$ wynosi

$$d_{\cos}(\mathbf{u}, \mathbf{v}) = 1 - \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2},$$

gdzie $\mathbf{u} \cdot \mathbf{v} = \sum_{k=1}^d u_k v_k$, a $\|\cdot\|_2$ jest normą euklidesową. Miara przyjmuje wartości w $[0, 2]$. $d_{\cos} = 0$ oznacza identyczny kierunek, $d_{\cos} = 1$ prostopadłość, natomiast $d_{\cos} = 2$ oznacza przeciwny kierunek. W kontekście reprezentacji ukrytych $\mathbf{h}_t^{(l_i)} \in \mathbb{R}^d$ odległość cosinusowa mierzy zmianę kierunku wektora, niezależnie od jego normy. Jest więc odporna na skalowanie normy wywołane normalizacją warstw. Odległość cosinusowa służy do mierzenia, jak bardzo dana reprezentacja jest zbliżona do określonych kierunków odpowiadających wysokopoziomowym pojęciom (konceptom) [26].

3.3.3. Dryf międzywarstwowy

Dla $i = 1, \dots, L$ definiujemy *dryf międzywarstwowy* tokenu t jako odległość kosinusową między reprezentacją ukrytą warstwy l_{i-1} a warstwy l_i :

$$\text{layer_drift}_t^{(l_i)} = d_{\cos}(\mathbf{h}_t^{(l_{i-1})}, \mathbf{h}_t^{(l_i)}), \quad i = 2, \dots, L. \quad (3.10)$$

Dla pierwszej sondowanej warstwy ($i = 1$) cecha przyjmuje wartość 0, ponieważ warstwa l_0 (embedding) nie jest uwzględniana w sondowaniu.

Uzasadnienie. Przetwarzanie residual stream w transformerze można rozumieć jako serię addytywnych aktualizacji, które płynnie i stopniowo kumulują się w jego reprezentacji ukrytej [27]. Token, którego reprezentacja gwałtownie zmienia kierunek między sąsiednimi warstwami, jest przetwarzany w sposób nietypowy, ponieważ nowo dodany kontekst drastycznie dominuje nad dotychczasowymi informacjami. Postawiono hipotezę, że gwałtowna zmiana kierunku może sygnalizować niestabilność w procesie wnioskowania prowadzącą do powstawania pseudohalucynacji [15]. Opisana metoda stanowi rozszerzenie podejścia zaproponowanego przez Su i in. [15]. Jednak Su i in. nie badają różnic między warstwami, a jedynie wyznaczają średni wektor po wszystkich wektorach stanów ukrytych tokenów dla warstw.

3.3.4. Dryf tokena

Funkcja mierzy *różnice* w stanach ukrytych odpowiadających kolejnym tokenom w sekwencji. O ile dryf warstwowy (sekcja 3.3.3) porównuje sąsiednie warstwy dla tego samego tokena, ta cecha porównuje sąsiednie tokeny w tej samej warstwie. Odległość kosinusowa między kolejnymi tokenami wynosi w warstwie l_i :

$$\text{token_drift}_t^{(l_i)} = d_{\cos}(\mathbf{h}_{t-1}^{(l_i)}, \mathbf{h}_t^{(l_i)}) \quad (3.11)$$

gdzie dla pierwszego tokena odpowiedzi jako $\mathbf{h}_{t-1}^{(l_i)}$ przyjmuje się stan ukryty ostatniego tokena promptu.

Uzasadnienie. Wysoka wartość $\text{token_drift}_t^{(l_i)}$ oznacza dużą lokalną zmianę reprezentacji między sąsiednimi pozycjami. W przebiegu spójnej odpowiedzi reprezentacje kolejnych tokenów zmieniają się zazwyczaj płynnie. Postawiono hipotezę, że nietypowo duże wartości tej cechy mogą współwystępować z fragmentami odpowiedzi, w których model przechodzi do mniej stabilnego trybu generacji, potencjalnie powiązanego z pseudohalucynacją [15]. Opisana metoda stanowi rozszerzenie podejścia Su i in. [15]. Kluczowa innowacja polega na analizie lokalnego dryfu trajektorii pomiędzy każdą parą sąsiadujących tokenów, w przeciwieństwie do tamtego podejścia, gdzie ograniczono się do średniej po wszystkich tokenach i warstwach.

3.3.5. Entropia

Omawiana cecha mierzy *stopień rozproszenia* rozkładu prawdopodobieństwa w kolejnych warstwach modelu. Aby ograniczyć koszt obliczeniowy, projekcję wykonujemy wyłącznie na zbiorze top- K tokenów słownika wybranych na podstawie finalnego rozkładu prawdopodobieństwa (obliczanego z ostatniej warstwy l_L). W eksperymentach przyjęto $K = 1000$. Wartość ta jest typowym wyborem stosowanym w metodach opartych na analizie logitów. Przy zbyt małym K renormalizowany rozkład pomija istotną część masy prawdopodobieństwa znajdującej się poza ścisłą czołówką kandydatów, natomiast zbyt duże K niepotrzebnie zwiększa koszt obliczeniowy bez znaczącego wzbogacenia informacji o rozkładzie. Pełny wektor logitów dla tokenu na pozycji t z warstwy l_i wylicza się z następującego wzoru:

$$\tilde{\mathbf{z}}_t^{(l_i)} = \mathbf{h}_{t-1}^{(l_i)} W_U^\top \in \mathbb{R}^{|\mathcal{V}|}, \quad (3.12)$$

gdzie indeks $t-1$ wynika z autoregresyjnej natury modelu. Stan ukryty na pozycji $t-1$ wyznacza rozkład prawdopodobieństwa tokenu na pozycji t . Zbiór top- K kandydatów wyznaczamy z finalnej warstwy i oznaczamy jako $\mathcal{K}_t^{\text{ent}}$ a następnie ograniczamy wektor logitów warstwy l do tego zbioru:

$$\mathbf{z}_t^{(l_i)} = (\tilde{\mathbf{z}}_t^{(l_i)})_{\mathcal{K}_t^{\text{ent}}} \in \mathbb{R}^K. \quad (3.13)$$

Prawdopodobieństwa kandydatów w warstwie l_i są renormalizowane w obrębie zbioru $\mathcal{K}_t^{\text{ent}}$:

$$\tilde{p}_{t,k}^{(l_i)} = \frac{\exp(z_{t,k}^{(l_i)})}{\sum_{j \in \mathcal{K}_t^{\text{ent}}} \exp(z_{t,j}^{(l_i)})}, \quad k \in \mathcal{K}_t. \quad (3.14)$$

Ponieważ softmax jest liczony po ograniczeniu do $\mathcal{K}_t^{\text{ent}}$, wartości $\tilde{p}_{t,k}^{(l_i)}$ są prawdopodobieństwami renormalizowanymi w obrębie top- K , a nie pełnymi prawdopodobieństwami nad całym słownikiem $\mathcal{V}$. Dla odróżnienia od pełnego rozkładu softmax nad słownikiem $\mathcal{V}$ (stosowanego w sekcji 2.1.8), renormalizowany rozkład oznaczamy tyldą $\tilde{p}$. Znormalizowaną entropię Shannona rozkładu renormalizowanego na top- K definiujemy jako:

$$\text{entropy}_t^{(l_i)} = -\frac{1}{\log K} \sum_{k \in \mathcal{K}_t^{\text{ent}}} \tilde{p}_{t,k}^{(l_i)} \log \tilde{p}_{t,k}^{(l_i)}, \quad (3.15)$$

gdzie czynnik $1/\log K$ normalizuje entropię do przedziału $[0, 1]$ (entropia maksymalna osiągnięta jest dla rozkładu jednostajnego nad K elementami).

Uzasadnienie. Niska wartość $\text{entropy}_t^{(l_i)}$ w warstwach końcowych wskazuje na koncentrację masy rozkładu $\mathbf{p}^{(l_i)}$ w otoczeniu pojedynczego kandydata. W przypadku gdy $\text{entropy}_t^{(l_i)}$ pozostaje wysokie w końcowych warstwach, rozkład $\mathbf{p}^{(l_i)}$ jest bliski jednostajnemu na zbiorze top- K tokenów, co oznacza brak jednoznacznej preferencji modelu względem generowanego tokenu. Opisywana metoda jest zaczerpnięta z pracy Sharma i in. [20]. Jedyną modyfikacją jest wprowadzenie dzielenia przez czynnik $\log K$, co pozwala na lepszą interpretację skali

obserwowanego zjawiska.

3.3.6. Niepewność

Niepewność tokenu t w warstwie l_i definiujemy jako dopełnienie do jedności maksymalnego prawdopodobieństwa w rozkładzie renormalizowanym na zbiorze top- K kandydatów:

$$\text{unc}_t^{(l_i)} = 1 - \max_{k \in \mathcal{K}_t^{\text{ent}}} \tilde{p}_{t,k}^{(l_i)}, \quad (3.16)$$

gdzie $\tilde{p}_{t,k}^{(l_i)}$ zdefiniowano w równaniu (3.14). Miara $\text{unc}_t^{(l_i)}$ informuje o braku dominującego kandydata w zbiorze top- K , ponieważ $\tilde{p}_{t,k}^{(l_i)}$ są renormalizowane w obrębie top- K . Wartość $\text{unc}_t^{(l_i)}$ nie mierzy całkowitej masy prawdopodobieństwa poza tym zbiorem.

Uzasadnienie. Postawiono hipotezę, że tokeny generowane halucynacyjnie charakteryzują się podwyższoną wartością $\text{unc}_t^{(l_i)}$, odzwierciedlającą brak dominującego kandydata w rozkładzie predykcyjnym modelu. Wartość $\text{unc}_t^{(l_i)}$ bliska zeru wskazuje, że model skupia niemal całą masę prawdopodobieństwa na jednym kandydacie. Wysoka wartość $\text{unc}_t^{(l_i)}$ oznacza, że prawdopodobieństwo jest rozproszone równomiernie między kandydatami. Miara ta jest ściśle powiązana z entropią (sekcja 3.3.5), jednak bardziej wrażliwa na obecność dominującego kandydata. Entropia pozostaje wysoka nawet wtedy, gdy jeden token ma prawdopodobieństwo wyraźnie wyższe od pozostałych, lecz wciąż dalekie od jedności, podczas gdy $\text{unc}_t^{(l_i)}$ bezpośrednio odzwierciedla jego dominację. Miara ta nie stanowi oryginalnej propozycji autora, lecz adaptacji miary pewności opartej na maksymalnym prawdopodobieństwie softmax.

3.3.7. Odległość kontekstowa

Cecha mierzy odległość semantyczną tokenu od jego lokalnego kontekstu w przestrzeni ukrytej. Okno kontekstowe tokenu t o rozmiarze w definiujemy jako co najwyżej w tokenów bezpośrednio przed t

$$\text{cw}(t, w) = \{\tau \in \{q, \dots, T-1\} \mid t - w \leq \tau < t\}. \quad (3.17)$$

Okno zawiera co najwyżej w tokenów. Dla tokenów w pobliżu granic sekwencji okno jest obcinane do dostępnych indeksów odpowiedzi. Dla tokenu t przyjmujemy okno kontekstowe $\text{cw}(t, w)$ o rozmiarze $w = 3$. Wartość ta została wyznaczona empirycznie i przyjęta ze względu na największą skuteczność.

Centroid okna w warstwie l_i definiujemy jako

$$\bar{\mathbf{h}}_{\text{cw}}^{(l_i)}(t) = \frac{1}{|\text{cw}(t, w)|} \sum_{\tau \in \text{cw}(t, w)} \mathbf{h}_{\tau}^{(l_i)}, \quad (3.18)$$

a następnie odległość kosinusową tokenu t od centroidu

$$\text{context_dist}_t^{(l_i)} = d_{\cos}(\mathbf{h}_t^{(l_i)}, \bar{\mathbf{h}}_{\text{cw}}^{(l_i)}(t)). \quad (3.19)$$

Cecha ta różni się od `token_drift` zakresem porównania. `token_drift` mierzy odległość tokenu t od jednego bezpośredniego poprzednika $t-1$ i jest szczególnie czuły na nagłe zmiany kierunku reprezentacji w pojedynczym kroku sekwencji. `context_dist` porównuje natomiast token t z centroidem okna trzech poprzednich tokenów, a uśrednienie po w poprzednikach wygładza pojedyncze zmiany kierunku i mierzy izolację geometryczną tokenu od szerszego lokalnego sąsiedztwa. Obie cechy są komplementarne i uchwytyują różne aspekty niespójności sekwencji w przestrzeni ukrytej.

Uzasadnienie. Postawiono hipotezę, że token generowany halucynacyjnie jest geometrycznie izolowany od swojego lokalnego kontekstu w przestrzeni ukrytej, a wysoka wartość $\text{context_dist}_t^{(l_i)}$ może odzwierciedlać semantyczne zerwanie ciągłości w generowanej odpowiedzi [15]. Centroid (3.18) reprezentuje lokalną średnią fragmentu odpowiedzi przed tokenem t . Wartość $\text{context_dist}_t^{(l_i)}$ bliska zeru świadczy o podobieństwie reprezentacji tokenu do otoczenia. Natomiast wysokie wartości tej cechy wskazują na silną geometryczną izolację w przestrzeni ukrytej.

3.3.8. Rozproszenie predykcyjne

Cecha mierzy stopień rozproszenia kandydatów rozważanych przez model przy wyborze tokenu t . Rozkład $p^{(l_i)}$ pochodzi z warstwy l_i opisanej w równaniu (3.13). Niech $\mathcal{K}_t^{\text{spr}}$ oznacza zbiór K tokenów o najwyższym prawdopodobieństwie w rozkładzie $p^{(l_i)}$. W przypadku tego eksperymentu wybrano wartość $K = 50$, oparte na eksperymencie zamieszczonym w dodatku A.2. Prawdopodobieństwa kandydatów normalizujemy do sumy 1 w obrębie zbioru $\mathcal{K}_t^{\text{spr}}$ za pomocą zależności

$$\hat{p}_{t,k}^{(l_i)} = \frac{p_{t,k}^{(l_i)}}{\sum_{k' \in \mathcal{K}_t^{\text{spr}}} p_{t,k'}^{(l_i)}}, \quad k \in \mathcal{K}_t^{\text{spr}}. \quad (3.20)$$

Kierunek odpowiadający tokenowi k określa wektor embeddingu wyznaczony z macierzy odoznaczania $W_U \in \mathbb{R}^{|\mathcal{V}| \times d}$

$$\mathbf{e}_k = W_U[k, :] \in \mathbb{R}^d. \quad (3.21)$$

Rozproszenie predykcyjne definiujemy jako sumę odległości kosinusowych obliczonych dla wszystkich unikalnych par różnych tokenów ze zbioru $\mathcal{K}_t^{\text{spr}}$. Każda odległość zostaje przeskalowana iloczynem znormalizowanych prawdopodobieństw obu porównywanych kandydatów. Ostateczna wartość cechy przyjmuje postać

$$\text{prediction_spread}_t^{(l_i)} = \sum_{\{j,k\} \subset \mathcal{K}_t^{\text{spr}}, j \neq k} \hat{p}_{t,j}^{(l_i)} \hat{p}_{t,k}^{(l_i)} d_{\cos}(\mathbf{e}_j, \mathbf{e}_k). \quad (3.22)$$

Uzasadnienie. Stan ukryty $\mathbf{h}_t \in \mathbb{R}^d$ koduje reprezentację semantyczną kontekstu w pozycji t . Logit tokenu k obliczany jest jako iloczyn skalarny

$$z_{t,k} = \mathbf{h}_t \cdot W_U[k, :],$$

gdzie $W_U[k, :] \in \mathbb{R}^d$ jest wierszem macierzy odwzorowującej stany ukryte na słownik. Iloczyn skalarny jest duży wtedy, gdy $W_U[k, :]$ i $\mathbf{h}_t$ wskazują zbliżony kierunek, dlatego token k uzyskuje wysokie prawdopodobieństwo dokładnie wtedy, gdy stan ukryty jest dobrze dopasowany do odpowiadającego mu wiersza w W_U .

Cecha $\text{prediction_spread}_t^{(l_i)}$ mierzy sumę odległości kosinusowych obliczonych dla unikalnych par wierszy $W_U[k, :]$ reprezentujących K kandydatów o najwyższym prawdopodobieństwie. Każda odległość między parą tokenów zostaje dodatkowo przeskalowana iloczynem prawdopodobieństw przypisanych tym elementom. Niska wartość oznacza, że wiersze wszystkich prawdopodobnych kandydatów wskazują zbliżony kierunek, a stan ukryty jednoznacznie aktywuje spójny pod względem semantycznym zbiór opcji.

Postawiono hipotezę, że wysoka wartość $\text{prediction_spread}_t^{(l_i)}$, odzwierciedlająca silne rozbieżności kierunków między poszczególnymi kandydatami posiadającymi wysokie prawdopodobieństwo, może być sygnałem niestabilności generacji powiązanej z pseudohalucynacją.

3.3.9. Cechy złożone

Cechy proste opisane w poprzednich podsekcjach mierzą każdy sygnał niestabilności modelu osobno, bez uwzględnienia zależności między nimi. Ponieważ przed przekazaniem danych do klasyfikatora stosowana jest agregacja statystyczna (sekcja 3.2.5), informacja o jednoczesnym współwystępowaniu dwóch sygnałów w obrębie tego samego tokenu i warstwy zostałaby utracona. Aby ją zachować, wprowadza się dwie cechy złożone konstruowane jako iloczyny wybranych cech prostych obliczane na poziomie tokenu i warstwy, przed agregacją. Składowe cech złożonych, tj. `unc`, `layer_drift` i `token_drift`, posiadają różne zakresy wartości. Aby każda ze składowych wносиła równorzędny wkład do iloczynu niezależnie od swojej skali, przed obliczeniem iloczynów stosuje się przeskalowanie metodą `RobustScaler`. Parametry transformacji, tj. mediana oraz rozstęp międzykwartylowy (IQR), są wyznaczone wyłącznie na zbiorze treningowym. Następnie te same parametry wykorzystuje się do przeskalowania danych testowych, co eliminuje ryzyko wycieku informacji między zbiorami. Przeskalowane wartości oznaczone są $\widetilde{}$ i służą wyłącznie do obliczenia iloczynów zdefiniowanych poniżej.

Niepewność i dryf warstwowy

Cecha ta pozwala zbadać wzajemną zależność pomiędzy niepewnością wyboru danego tokenu a dynamiką zmian kierunku jego reprezentacji między sąsiednimi warstwami modelu. Niech $\widetilde{\text{unc}}_t^{(l_i)}$ i $\widetilde{\text{layer_drift}}_t^{(l_i)}$ oznaczają odpowiednio cechy `unc` i `layer_drift` po przeskalowaniu opisanym powyżej. Cechę złożoną definiuje się jako

$$\text{unc_layer_drift}_t^{(l_i)} = \widetilde{\text{unc}}_t^{(l_i)} \cdot \widetilde{\text{layer_drift}}_t^{(l_i)}, \quad (3.23)$$

gdzie $\text{layer_drift}_t^{(l_i)}$ to dryf warstwowy z równania (3.10), a $\text{unc}_t^{(l_L)}$ to niepewność ostatniej warstwy z równania (3.16). Cecha $\text{unc_layer_drift}_t^{(l_i)}$ przyjmuje wysoką wartość, gdy model jest jednocześnie niepewny wyboru tokenu t oraz reprezentacja tego tokenu gwałtownie zmienia kierunek między warstwami l_i a l_{i+1} . Iloczyn łączy sygnał z przestrzeni logitów (niepewność) z sygnałem z przestrzeni ukrytej (dryf warstwowy). Postawiono hipotezę, że wysokie wartości tej cechy stanowią wskaźnik pseudohalucynacji. Cecha osiąga duże wartości wtedy, gdy jednocześnie obserwowana jest wysoka niepewność predykcyjna oraz duża zmiana reprezentacji między kolejnymi warstwami, co może świadczyć o niestabilnym procesie generowania [15].

Niepewność i dryf tokenowy

Cecha ta pozwala zbadać zależność pomiędzy niepewnością wyboru danego tokenu a dynamiką zmian jego stanu ukrytego wzdłuż kolejnych kroków sekwencji. Niech $\widehat{\text{unc}}_t^{(l_i)}$ i $\widehat{\text{token_drift}}_t^{(l_i)}$ oznaczają odpowiednio cechy `unc` i `token_drift` po przeskalowaniu opisanym powyżej. Cechę złożoną definiuje się jako

$$\text{unc_token_drift}_t^{(l_i)} = \widehat{\text{unc}}_t^{(l_L)} \cdot \widehat{\text{token_drift}}_t^{(l_i)}. \quad (3.24)$$

Cecha $\text{unc_token_drift}_t^{(l_i)}$ łączy niepewność wyjściową $\widehat{\text{unc}}_t^{(l_L)}$ wyznaczaną w przestrzeni logitów ostatniej warstwy z dryfem tokenowym $\widehat{\text{token_drift}}_t^{(l_i)}$ mierzonym geometrycznie w przestrzeni ukrytej jako odległość między $\mathbf{h}_t^{(l_i)}$ a $\mathbf{h}_{t-1}^{(l_i)}$. Postawiono hipotezę, że wysokie wartości tej cechy stanowią wskaźnik pseudohalucynacji. Cecha osiąga duże wartości wtedy, gdy jednocześnie obserwowana jest wysoka niepewność predykcyjna oraz duża lokalna zmiana reprezentacji, co może świadczyć o niestabilnym procesie generowania [15].

3.3.10. Wybór statystyk

Wybór trzech statystyk agregujących jest motywowany postawionymi hipotezami dotyczącymi natury pseudohalucynacji. Sygnały niestabilności mogą być obserwowane na różne sposoby wzdłuż sekwencji. Średnia mogłaby wykryć przypadek, w którym cała odpowiedź charakteryzuje się podwyższonym poziomem danego sygnału, co odpowiadałoby globalnej niestabilności generacji. Maksimum mogłoby natomiast uchwycić lokalną, chwilową niestabilność w pojedynczym tokenie, która mogłaby zostać zatarta przez uśrednienie. Odchylenie standardowe mogłoby z kolei mierzyć wewnętrzną zmienność sygnału wzdłuż sekwencji, uchwytując odpowiedzi, w których poszczególne tokeny wykazują silnie zróżnicowane wartości cechy.

3.3.11. Podsumowanie cech

Tabela 3.2 zestawia wszystkie badane cechy bazowe, wskazując przestrzeń modelu, w której są obliczane oraz co mierzą. Wszystkie cechy w tabeli 3.2 obliczane są wyłącznie na podstawie wewnętrznych sygnałów modelu w trakcie generowania odpowiedzi. Żadna z nich nie wymaga dostępu do odpowiedzi referencyjnej ani do zewnętrznej bazy wiedzy.

Cecha	Przestrzeń	Co mierzy
entropy	logity, top- K	Entropię logitów dla top- K tokenów.
unc	logity, top- K	Brak dominującego kandydata w top- K tokenów.
layer_drift	stany ukryte	Zmianę kierunku reprezentacji między sąsiednimi warstwami.
token_drift	stany ukryte	Lokalną zmianę reprezentacji między sąsiednimi pozycjami.
context_dist	stany ukryte	Odległość od lokalnego centroidu okna.
prediction_spread	W_U	Rozproszenie kandydatów w przestrzeni odosadzania.
unc_layer_drift	logity, stany ukryte	Iloczyn niepewności wyjściowej i dryfu warstwowego.
unc_token_drift	logity, stany ukryte	Iloczyn niepewności wyjściowej i dryfu tokenowego.

Tabela 3.2. Podsumowanie cech bazowych.

3.4. Klasyfikator

Ta sekcja skupia się na dokładnym opisie klasyfikatora zastosowanego do przeprowadzenia przedstawionych w pracy badań. Celem opisywanego klasyfikatora jest podjęcie decyzji, czy wygenerowana odpowiedź $\hat{a}_i$ jest pseudohalucynacją, wyłącznie na podstawie sygnałów wewnętrznych modelu językowego zebranych podczas przetwarzania pary $(q_i, \hat{a}_i)$.

3.4.1. Obsługa nierówności klas w klasyfikatorach

We wszystkich zbiorach danych występuje wyraźna nierównowaga klas między klasą poprawną ($\tilde{y}_i = 0$) a pseudohalucynacjami ($\tilde{y}_i = 1$). Aby zniwelować negatywny wpływ tego asymetrycznego rozkładu na proces trenowania, wprowadzono wagę dla klasy pozytywnej:

$$\gamma = \frac{n_0}{n_1}, \quad (3.25)$$

gdzie n_0 i n_1 oznaczają odpowiednio liczby próbek klasy $\tilde{y}_i = 0$ oraz $\tilde{y}_i = 1$ w zbiorze treningowym. Klasa $\tilde{y}_i = 0$ otrzymuje wagę równą 1.

Parametr γ jest wyznaczany na podstawie rozkładu klas w zbiorze treningowym i przekazywany bezpośrednio jako waga klasy pseudohalucynacji, tj. jako parametr `class_weight` w implementacji regresji logistycznej.

Tabela 3.3 przedstawia rozkład pseudoetykiet po etykietowaniu dla każdego ze zbiorów danych.

3.4.2. Regresja logistyczna

W badaniach wykorzystano model regresji logistycznej, którego implementacja pochodzi z biblioteki *Scikit-learn* [25]. Dla zagregowanego wektora cech $\tilde{\mathbf{f}}_i \in \mathbb{R}^{3D}$ przewidywane

Zbiór	Łącznie	Poprawne	Pseudohalucynacje
TriviaQA	10 000	41,9%	58,1%
SQuAD	10 000	41,7%	58,3%
NQOpen	10 000	15,8%	84,2%

Tabela 3.3. Rozkład pseudoetykiet po etykietowaniu metodami BLEURT-20 i ROUGE-L. Podział 80/20 zachowuje te proporcje w obu podzbiorach (podział stratyfikowany).

prawdopodobieństwo pseudohalucynacji wynosi:

$$\hat{p} = \frac{1}{1 + \exp(-\mathbf{w}^\top \tilde{\mathbf{f}}_i - w_0)}, \quad (3.26)$$

gdzie $\mathbf{w} \in \mathbb{R}^{3D}$ to wektor wag, a $w_0 \in \mathbb{R}$ to wyraz wolny. Parametry modelu wyznaczone są przez minimalizację ważonej entropii krzyżowej z regularyzacją ℓ_1 :

$$\mathcal{L}(\mathbf{w}, w_0) = \frac{1}{S} \sum_{i=1}^n s_i \left[-\tilde{y}_i \log \hat{p}_i - (1 - \tilde{y}_i) \log(1 - \hat{p}_i) \right] + \frac{\|\mathbf{w}\|_1}{SC}, \quad (3.27)$$

gdzie $s_i \in \{1, \gamma\}$ to waga i -tej próbki wyznaczana przez mechanizm `class_weight` ($s_i = \gamma$ gdy $\tilde{y}_i = 1$, $s_i = 1$ gdy $\tilde{y}_i = 0$), $S = \sum_{i=1}^n s_i$ to suma wag próbek, γ to waga klasy pseudohalucynacji zdefiniowana w równaniu (3.25), $\|\mathbf{w}\|_1 = \sum_{k=1}^{3D} |w_k|$ oznacza normę ℓ_1 wektora wag, a $C > 0$ to odwrotność siły regularyzacji. Postać odpowiada funkcji celu minimalizowanej przez tę implementację.

Hiperparametry modelu dobierano metodą przeszukiwania siatki (*grid search*) z wykorzystaniem 5-krotnej walidacji krzyżowej (*5-fold cross-validation*) przeprowadzonej na zbiorze treningowym. Przeszukiwano wartości parametru regularyzacji $C \in \{0,01, 0,1, 0,5\}$, typ regularyzacji (ℓ_1 lub ℓ_2) oraz solwery `liblinear`, `lbfgs`, `newton-cg` i `saga`. Konfigurację modelu wybierano na podstawie najwyższej średniej wartości AUROC uzyskanej w fazie walidacji krzyżowej.

Najlepsze wyniki uzyskano dla regularyzacji ℓ_1 , parametru $C = 0,1$ oraz solvera `liblinear`. Maksymalną liczbę iteracji ustawiono na 10 000. Wybrana wartość C odpowiada stosunkowo silnej regularyzacji, ograniczającej nadmierne dopasowanie modelu do danych treningowych.

3.4.3. Protokół treningu i walidacji

Dla każdego zbioru pytań pobrano 10 000 pierwszych par pytanie-odpowiedź (według kolejności w zbiorze na platformie Hugging Face), którą podzielono losowo w stosunku 80/20 na zbiór treningowy i testowy przy użyciu stratyfikowanego podziału zachowującego proporcje klas (`random_state=42`). `RobustScaler` (równanie (3.9)) dopasowywany jest wyłącznie na danych treningowych i stosowany do zbioru testowego, co zapobiega wyciekowi informacji (ang. *data leakage*). Algorytm 3 podsumowuje trening i ewaluację.

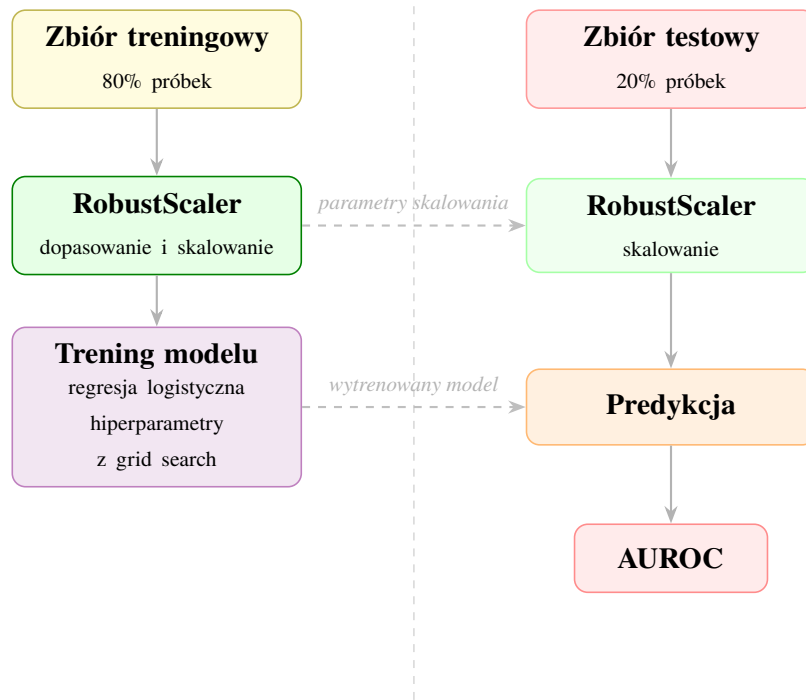
Algorytm 3 Uczenie i ewaluacja regresji logistycznej

Require: macierze cech $\{\Phi_i\}$, pseudoetykiety $\{\tilde{y}_i\}$

Ensure: AUROC

- 1: Agreguj $\Phi_i \rightarrow \mathbf{f}_i \in \mathbb{R}^{3D}$ dla wszystkich próbek
 - 2: Podziel dane na $\mathcal{D}_{tr}$ i $\mathcal{D}_{te}$ (stratyfikowany podział 80/20)
 - 3: $\gamma \leftarrow |\mathcal{D}_{tr}^{(0)}| / |\mathcal{D}_{tr}^{(1)}|$ ▷ stosunek liczebności klas w zbiorze treningowym
 - 4: Dopasuj RobustScaler wyłącznie na $\mathcal{D}_{tr}$, zastosuj transformację do $\mathcal{D}_{tr}$ i $\mathcal{D}_{te}$
 - 5: Dobierz hiperparametry (C , regularyzacja, solver) metodą *grid search* z 5-krotną walidacją krzyżową na $\mathcal{D}_{tr}$, kryterium: średnie AUROC z CV
 - 6: Wytrenuj regresję logistyczną na pełnym $\mathcal{D}_{tr}$ z najlepszymi hiperparametrami i wagą $\{0 : 1, 0, 1 : \gamma\}$
 - 7: Wyznacz AUROC na $\mathcal{D}_{te}$
-

Wytrenowany model ewaluowany jest na wydzielonym zbiorze testowym. Kompletny potok treningu i walidacji przedstawia rysunek 3.2.



Rysunek 3.2. Potok treningu i walidacji regresji logistycznej. Lewa kolumna odpowiada fazie treningu, prawa fazie ewaluacji. RobustScaler dopasowywany jest wyłącznie na danych treningowych, a jego parametry przenoszone są do transformacji zbioru testowego. Zbiór testowy wykorzystywany jest jednokrotnie, wyłącznie do ewaluacji końcowej.

Rozdział 4

Wyniki badań i ich analiza

4.1. Opis eksperymentu

Główną miarą skuteczności używaną w tym eksperymencie jest AUROC (ang. *Area Under the ROC Curve*) [28]. Dla klasyfikatora binarnego z progiem decyzyjnym $\tau \in \mathbb{R}$ wprowadza się czułość $TPR(\tau)$ oraz odsetek fałszywych alarmów $FPR(\tau)$

$$TPR(\tau) = \frac{TP(\tau)}{TP(\tau) + FN(\tau)}, \quad (4.1)$$

$$FPR(\tau) = \frac{FP(\tau)}{FP(\tau) + TN(\tau)}, \quad (4.2)$$

gdzie $TPR(\tau)$ oznacza odsetek pseudohalucynacji poprawnie wykrytych, a $FPR(\tau)$ odsetek odpowiedzi poprawnych błędnie sklasyfikowanych jako pseudohalucynacje. Krzywa ROC jest wykresem punktów $(FPR(\tau), TPR(\tau))$ dla τ zmiennego w $\mathbb{R}$, a AUROC jest polem powierzchni pod tą krzywą. Formalnie, dla klasyfikatora zwracającego wynik $s \in \mathbb{R}$ oraz etykiety $\tilde{y}_i \in \{0, 1\}$,

$$AUROC = P(s(x^+) > s(x^-)) + \frac{1}{2} P(s(x^+) = s(x^-)), \quad (4.3)$$

gdzie x^+ i x^- są niezależnymi losowymi próbkami odpowiednio klasy pozytywnej (pseudohalucynacja) i negatywnej (odpowiedź poprawna). AUROC wyraża prawdopodobieństwo, że losowo wybrany przykład pseudohalucynacji otrzyma wyższy wynik od losowo wybranego przykładu poprawnej odpowiedzi. Wartość 0,5 odpowiada klasyfikatorowi losowemu, natomiast wartość 1,0 wyraża bezbłędną klasyfikację. Miara ta jest niezależna od progu decyzyjnego i mniej wrażliwa na niezbalansowanie klas niż zwykła dokładność (ang. *accuracy*), co czyni ją właściwym wyborem w niniejszym eksperymencie, gdzie proporcja pseudohalucynacji do poprawnych odpowiedzi jest nierówna.

4.2. Wyniki klasyfikacji

Tabela 4.1 przedstawia wyniki klasyfikacji pseudohalucynacji uzyskane dla modelu Llama-3.1-8B na zbiorach TriviaQA, SQuAD oraz NQOpen.

W celu oceny stabilności wyników wyznaczono 95% przedziały ufności dla miary AUROC z wykorzystaniem $B = 100\,000$ losowań bootstrapowych ze zwracaniem. Rozmiar pojedynczego resamplingu ustalono na $N = 2000$, co stanowi wielkość zbioru testowego.

Uzyskane wartości AUROC wskazują na wysoką skuteczność klasyfikatora w identyfikacji pseudohalucynacji we wszystkich analizowanych zbiorach danych. Jednocześnie relatywnie wąskie przedziały ufności świadczą o stabilności wyników i sugerują, że badane cechy zawierają informacje umożliwiające skuteczne wykrywanie pseudohalucynacji.

Metryka	TriviaQA	SQuAD	NQOpen
AUROC	0,897	0,883	0,831
95% CI	[0,8834 – 0,9107]	[0,8672 – 0,8973]	[0,8090 – 0,8530]

Tabela 4.1. Wyniki klasyfikacji pseudohalucynacji dla modelu Llama-3.1-8B wraz z 95% przedziałami ufności wyznaczonymi metodą bootstrapową.

4.3. Porównanie skuteczności z innymi metodami biało-skrzynkowymi

Proponowane podejście klasyfikuje pseudohalucynacje wyłącznie na podstawie interpretowalnych cech wewnętrznych modelu. Należy podkreślić, że bezpośrednie porównanie wartości AUROC między pracami jest obarczone istotnymi ograniczeniami. Poszczególne prace korzystają z różnych modeli językowych, różnych protokołów podziału na zbiór treningowy i testowy oraz różnych sposobów ustalania poprawności odpowiedzi wygenerowanej przez model. Porównanie ma zatem charakter orientacyjny i służy jedynie jako punkt odniesienia. Żadna z metod referencyjnych nie była uruchamiana ponownie w warunkach eksperymentalnych niniejszej pracy.

Metoda	Model	TriviaQA	SQuAD	NQOpen
Perplexity	Llama-3.1-8B	0,763	n/d	0,503
HaloScope	Llama-3.1-8B	0,762	0,840	0,627
TOHA	Llama-3.1-8B	n/d	0,870	n/d
LapEigvals	Llama-3.1-8B	0,889	n/d	0,827
HARP	Llama-3.1-8B	0,929	n/d	0,894
Niniejsza praca	Llama-3.1-8B	0,897	0,883	0,831

Tabela 4.2. Tabela porównawcza wartości wskaźnika AUROC uzyskanych przez wybrane metody białoskrzynkowe. Ewaluację przeprowadzono na zbiorach danych pochodzących z tych samych benchmarków co wykorzystane w niniejszym badaniu, jednak nie były to tożsame instancje danych. Wartości referencyjne dla metody TOHA zaczerpnięto z pracy Bazarovej i in. [18], w której zaprezentowano tę procedurę. Z kolei wyniki dla metryki perplexity oraz systemu HaloScope przytoczono za publikacją Hu i in. [16].

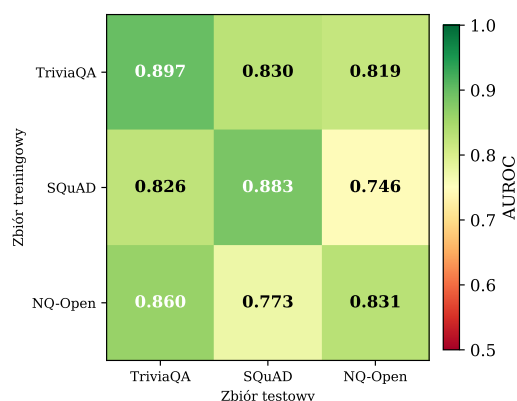
Tabela 4.2 przedstawia orientacyjne zestawienie z wybranymi metodami detekcji pseudohalucynacji raportującymi wyniki na zbiorach TriviaQA, SQuAD i NQOpen. Nie jest to porównanie kontrolowane, gdyż poszczególne metody różnią się modelem bazowym, sposobem pseudoetykietowania, procedurą podziału danych oraz zakresem dostępu do informacji wewnętrznych modelu. Wartości AUROC należy traktować jako punkt odniesienia, a nie jako dowód przewagi. Pełne porównanie wymagałoby uruchomienia wszystkich metod na tych samych danych. Uwzględniono metody oparte na perplexity [19], HaloScope [24], TOHA [18], LapEigvals [17] oraz HARP [16] (HARP operuje na surowych stanach ukrytych, nie na wyekstrahowanych cechach jawnych).

Przeprowadzone eksperymenty wykazują, że uzyskane przez nas wartości osiągają poziom

konkurencyjny względem alternatywnych metod detekcji.

4.4. Badanie skuteczności metody na danych z różnych zbiorów danych

Chcąc ustalić, jak metoda radzi sobie z generalizacją, przetestowano ją w warunkach, gdy dane treningowe i testowe pochodziły z różnych źródeł. Rysunek 4.1 przedstawia wyniki uzyskane dla modelu Llama-3.1-8B i sugeruje zdolność do przenoszenia sygnału między zbiorami danych.



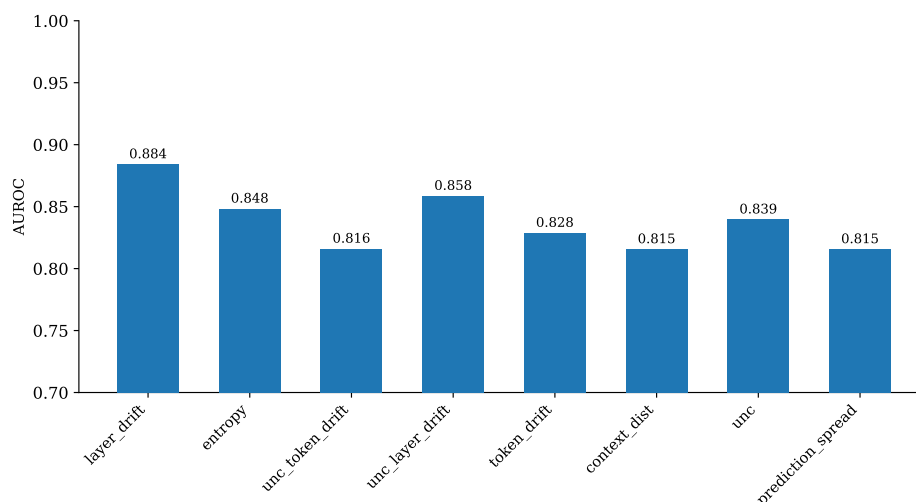
Rysunek 4.1. Macierz AUROC testu generalizacji między zbiorami danych dla modelu Llama-3.1-8B.

Regresja logistyczna osiąga wartości AUROC na przekątnej w przedziale od 0,831 do 0,897, przy czym najwyższy wynik uzyskano dla TriviaQA, a najniższy dla NQOpen. W scenariuszach krzyżowych skuteczność jest na ogół niższa, spadek AUROC pomiędzy wynikiem na przekątnej a wynikami krzyżowymi waha się od 0,058 do 0,137. Największy bezwzględny spadek skuteczności obserwuje się dla modelu trenowanego na NQOpen i testowanego na SQuAD (0,773) oraz dla modelu trenowanego na SQuAD i testowanego na NQOpen (0,746). Wynik ten wskazuje, że transfer między tymi zbiorami jest szczególnie trudny w obu kierunkach. Wyjątek stanowi kierunek NQOpen do TriviaQA, gdzie AUROC wynosi 0,860 i przekracza wynik na przekątnej NQOpen, co sugeruje, że cechy wewnętrzne modelu wyuczone na pytaniach otwartych pozostają równie skuteczne po przeniesieniu ich do zbioru TriviaQA. Wyniki sugerują, że wyekstrahowane cechy wewnętrzne modelu zachowują dobrą skuteczność niezależnie od źródła pytań.

4.5. Analiza cech

Tematem tej sekcji jest analiza skuteczności każdej z badanych cech rozpatrywanej osobno. Wszelkie badania przeprowadzono na modelu Llama-3.1-8B na zbiorze TriviaQA. Wyniki badania przedstawiono na rysunku 4.2.

Wszystkie badane cechy pojedyncze osiągają dla regresji logistycznej wartości AUROC powyżej 0,81, co świadczy o tym, że każdy z analizowanych sygnałów wewnętrznych



Rysunek 4.2. Wartość AUROC wyznaczona osobno dla każdej cechy na modelu regresji logistycznej.

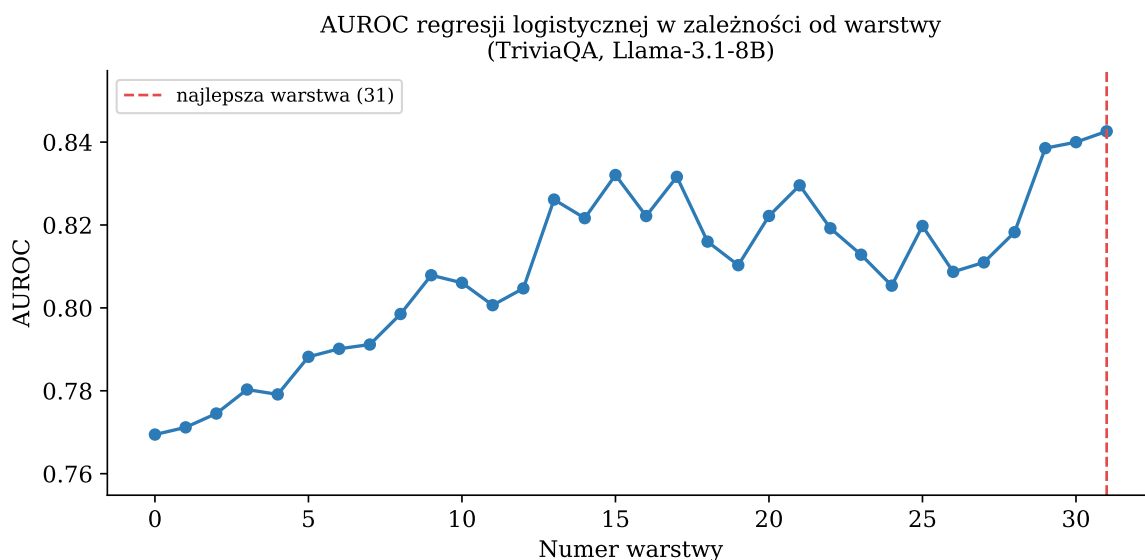
modelu niesie ze sobą istotną informację predykcyjną. Niemniej między cechami występują wyraźne różnice w skuteczności. Najwyższy wynik osiąga `layer_drift` (0,884), następnie `unc_layer_drift` (0,858), `entropy` (0,848), `unc` (0,839), `token_drift` (0,828), `unc_token_drift` (0,816), `context_dist` (0,815) oraz `prediction_spread` (0,815). Ogólny rozstęp między najlepszą a najslabszą cechą wynosi 0,069, co przy wysokim poziomie skuteczności cech wskazuje raczej na różnice w stopniu przydatności niż na wyraźną granicę między cechami użytecznymi a bezużytecznymi. Badania wykazują, że `token_drift` osiąga nieco wyższy AUROC (0,828) niż `context_dist` (0,815). Nagłe zmiany między sąsiednimi tokenami stanowią zatem silniejszy wskaźnik niestabilności procesu generacji niż geometryczna odległość od uśrednionego kontekstu, która okazała się sygnałem o mniejszej wartości diagnostycznej.

4.5.1. Wnioski

Uzyskane wyniki są zgodne z hipotezą, że wewnętrzne sygnały modelu językowego zawierają informację pozwalającą odróżnić odpowiedzi zgodne z referencją od pseudohalucynacji, niezależnie od tego, która konkretna cecha jest rozpatrywana. Fakt, że nawet najslabsza z badanych cech przekracza próg 0,81 AUROC, sugeruje, że zjawisko pseudohalucynacji pozostawia ślad w wielu różnych aspektach przetwarzania wewnętrznego modelu jednocześnie. Dominacja cechy `layer_drift` wskazuje, że dynamika zmian kierunku reprezentacji ukrytej między warstwami jest szczególnie czułym wskaźnikiem niestabilności generacji. Cechy złożone osiągają wyniki zbliżone do pozostałych badanych cech, jednak żadna z nich nie przewyższa wyniku swoich składowych rozpatrywanych osobno. Możliwym wyjaśnieniem jest to, że obie składowe każdej cechy złożonej rosną jednocześnie przy pseudohalucynacjach, wobec czego ich iloczyn skaluje wartości, lecz nie wnosi dodatkowego sygnału odróżniającego odpowiedzi poprawne od pseudohalucynacji względem tego, co niosą składowe z osobna.

4.6. Analiza warstwowa

Dotychczasowe eksperymenty agregowały cechy ze wszystkich warstw jednocześnie. Niniejsza sekcja zadaje pytanie o to, które warstwy wnoszą największy wkład do sygnału predykcyjnego, izolując cechy każdej warstwy z osobna. Eksperyment przeprowadzono na zbiorze TriviaQA. Trening i ewaluacje przeprowadzono identycznie jak w przypadku wyżej opisanych sekcji.



Rysunek 4.3. AUROC regresji logistycznej w zależności od warstwy (TriviaQA, Llama-3.1-8B).

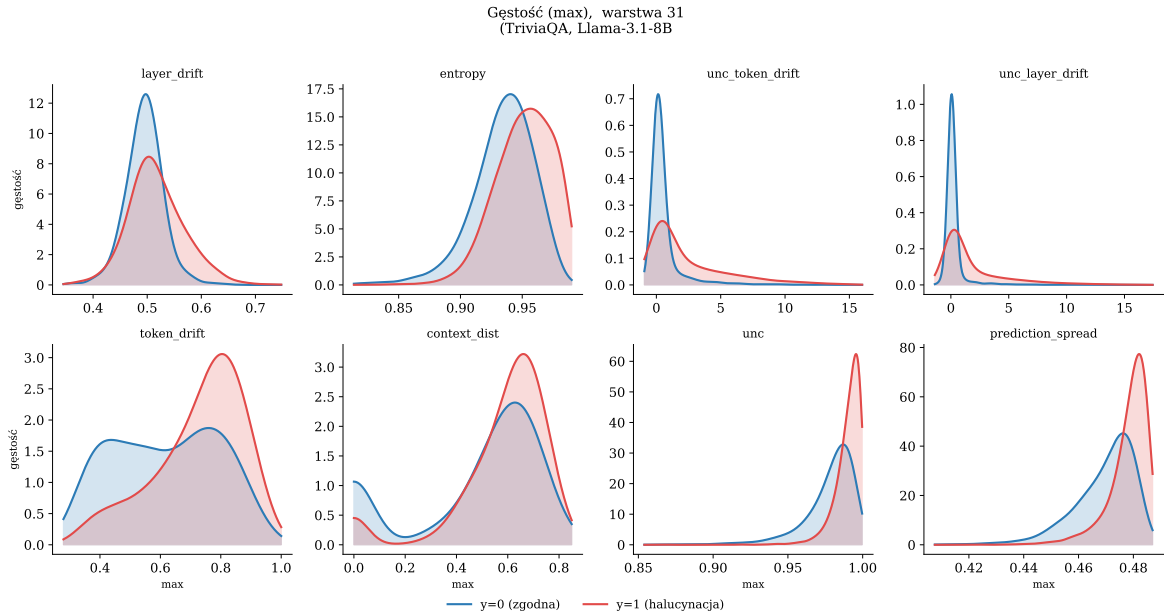
Rysunek 4.3 przedstawia AUROC dla poszczególnych warstw. Warstwy środkowe i końcowe modelu osiągają wyraźnie wyższy AUROC niż warstwy wejściowe, co sugeruje, że reprezentacje w tych warstwach silniej odzwierciedlają niepewność modelu względem generowanej odpowiedzi. Najlepsze AUROC wynoszące 0,84 osiągnięto dla warstwy ostatniej.

4.6.1. Analiza cech bazowych

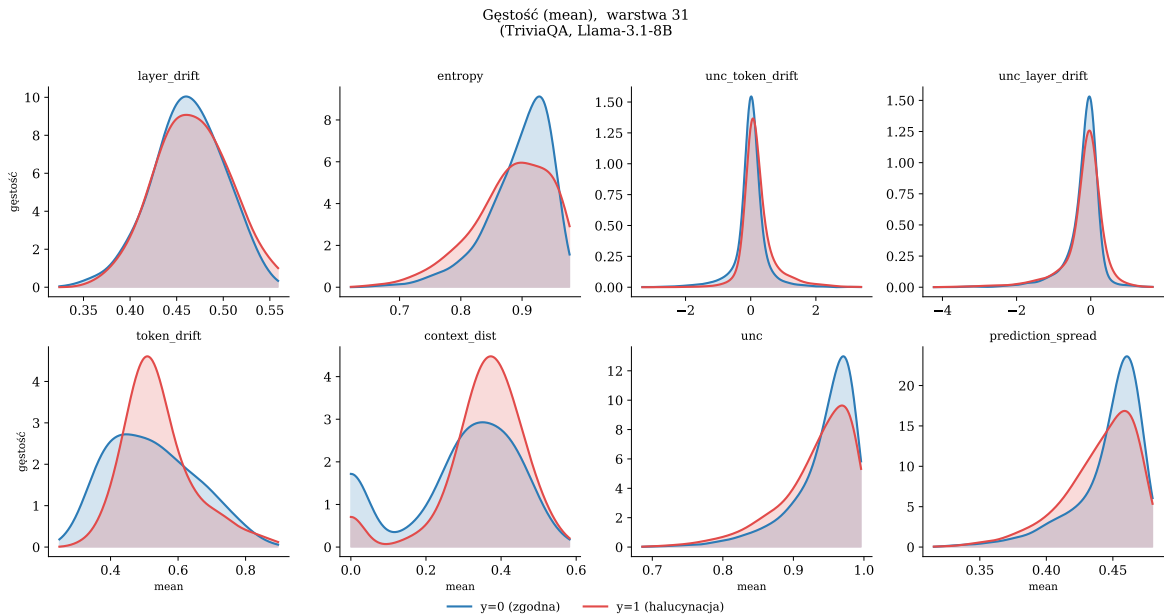
Dla warstwy o najwyższym AUROC wyznaczono funkcje gęstości ośmiu cech bazowych oddzielnie dla odpowiedzi zgodnych z referencją ($\tilde{y}_i=0$) i pseudohalucynacji ($\tilde{y}_i=1$). Analizę przeprowadzono dla trzech statystyk agregujących wartości po tokenach sekwencji, tj. maksimum, średniej i odchylenia standardowego, przedstawionych odpowiednio na rysunkach 4.4, 4.5 i 4.6. Do estymowania funkcji gęstości każdej cechy użyto `gaussian_kde` z biblioteki `scipy.stats`. Oś pionowa wykresów odpowiada wyestymowanej gęstości (pole pod każdą krzywą wynosi 1).

We wszystkich badanych cechach funkcje gęstości wartości maksymalnych wykazują przesunięcie klasy $\tilde{y}_i = 1$ w prawo względem klasy $\tilde{y}_i = 0$, co oznacza, że w sekwencjach pseudohalucynacji przynajmniej część tokenów osiąga wyższe wartości sygnałów wewnętrznych niż w odpowiedziach poprawnych. Zależność ta jest spójna dla wszystkich analizowanych cech.

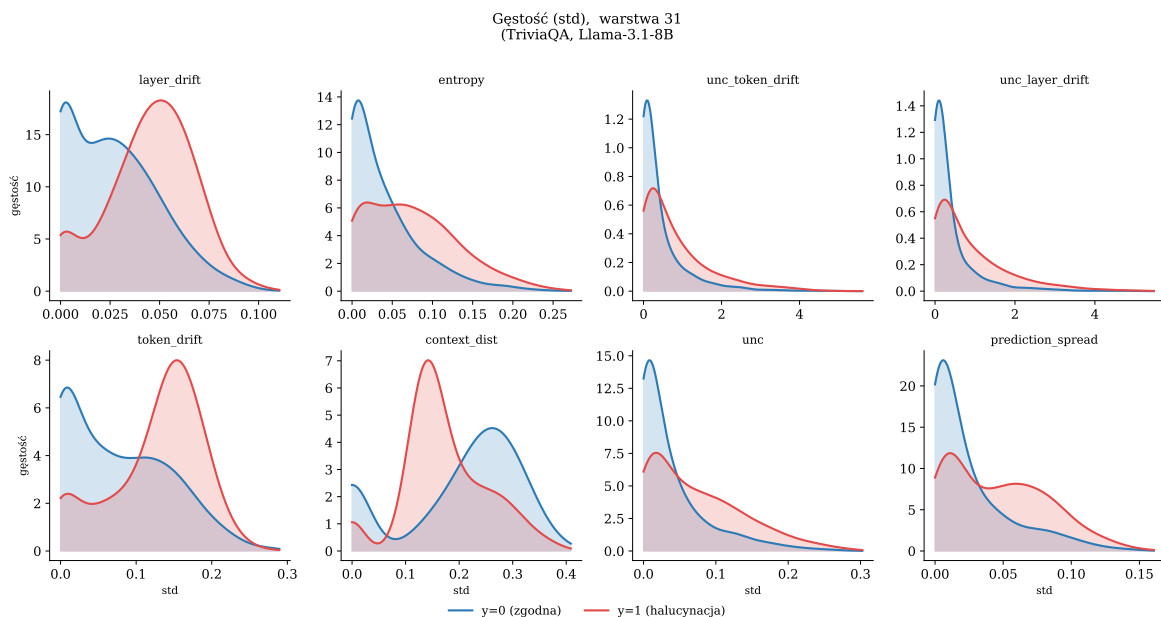
W przypadku wartości średnich separacja klas jest znacznie słabsza. Uśrednianie po tokenach wygładza lokalne różnice w sekwencji, przez co część informacji ulega zatarciu.



Rysunek 4.4. Rozkłady ośmiu cech bazowych dla najlepszej warstwy, statystyka maksimum po tokenach. Odpowiedzi zgodne z referencją (niebieski) i pseudohalucynacje (czerwony). TriviaQA, Llama-3.1-8B.



Rysunek 4.5. Rozkłady ośmiu cech bazowych dla najlepszej warstwy, statystyka średniej po tokenach. Odpowiedzi zgodne z referencją (niebieski) i pseudohalucynacje (czerwony). TriviaQA, Llama-3.1-8B.



Rysunek 4.6. Rozkłady ośmiu cech bazowych dla najlepszej warstwy, statystyka odchylenia standardowego po tokenach. Odpowiedzi zgodne z referencją (niebieski) i pseudohalucynacje (czerwony). TriviaQA, Llama-3.1-8B.

Bardzo dobrą separację zapewnia odchylenie standardowe. Pseudohalucynacje charakteryzują się wyższymi wartościami tej statystyki, co wskazuje na większą zmienność sygnałów w obrębie sekwencji. Odpowiedzi poprawne są natomiast bardziej jednorodne pod tym względem.

Należy zaznaczyć, że przedstawione krzywe gęstości odnoszą się do pojedynczej warstwy, podczas gdy klasyfikator wykorzystuje cechy zagregowane ze wszystkich warstw. Z tego powodu wizualna separacja nie musi bezpośrednio odpowiadać skuteczności klasyfikacji. Przykładem jest cecha `layer_drift`, która mimo najwyższego AUROC nie wykazuje szczególnie wyraźnej separacji w pojedynczej warstwie, co sugeruje, że jej informatywność wynika z efektu kumulacji między warstwami.

Spośród wszystkich cech najbardziej wyraźną separację wizualną wykazują `entropy` oraz `unc`, natomiast najślabszą `context_dist`, co jest zgodne z ich skutecznością klasyfikacyjną.

4.7. Podsumowanie wyników

Przeprowadzone eksperymenty pokazują, że połączenie wszystkich ośmiu cech prowadzi do poprawy jakości klasyfikacji w porównaniu z każdą z cech rozpatrywaną osobno. Regresja logistyczna trenowana na pełnym zestawie cech osiąga AUROC = 0,897 na zbiorze TriviaQA, 0,883 na SQuAD i 0,831 na NQOpen dla modelu Llama-3.1-8B (tabela 4.1). Wynik ten sugeruje, że poszczególne cechy uchwytyują komplementarne aspekty sygnału wewnętrznego modelu, które łącznie poprawiają zdolność klasyfikatora do odróżnienia odpowiedzi poprawnych od pseudohalucynacji.

Analiza pojedynczych cech wskazuje, że każda z nich osobno zachowuje wartość predykcyjną, przy czym `layer_drift` osiąga najwyższy wynik (0,884), a cechy `context_dist` i

`prediction_spread` najniższy (0,815). Rozstęp między najlepszą a najslabszą cechą wynosi zaledwie 0,069, co świadczy o tym, że żadna z cech nie dominuje w sposób bezwzględny nad pozostałymi.

Analiza funkcji gęstości cech dla wybranej warstwy modelu wskazuje, że różnice między klasami są widoczne na poziomie poszczególnych sygnałów wewnętrznych. Najbardziej wyraźną separację wykazują statystyki maksimum i odchylenia standardowego, przy czym pseudohalucynacje konsekwentnie osiągają wyższe wartości maksymalne oraz większą zmienność wewnętrzną sekwencji niż odpowiedzi poprawne. Analiza warstwowa wskazuje ponadto, że skuteczność poszczególnych cech rośnie wraz z głębokością sieci, co sugeruje, że reprezentacje z wyższych warstw niosą bogatszą informację o poprawności generowanej odpowiedzi.

Uzyskane wyniki są konkurencyjne względem metod referencyjnych zestawionych w tabeli 4.2. Spośród metod ewaluowanych na tym samym modelu LapEigvals osiąga AUROC 0,889 na TriviaQA i 0,827 na NQOpen, a TOHA uzyskuje 0,870 na SQuAD. Jednak sprawiedliwe porównanie wymagałoby uruchomienia metod w tych samych warunkach.

Badanie skuteczności metody na danych z różnych zbiorów danych (rysunek 4.1) pokazuje, że klasyfikator wytrenowany na jednym zbiorze pytań częściowo zachowuje skuteczność na danych z innego źródła, przy czym stopień degradacji jest zróżnicowany w zależności od kierunku transferu. Sugeruje to, że wyekstrahowane cechy uchwytują pewne ogólne właściwości sygnału wewnętrznego modelu, niezależne od konkretnego zbioru pytań.

Rozdział 5

Podsumowanie

Niniejsza praca badała hipotezę, że odpowiedzi o niskiej zgodności z referencją pozostawiają mierzalne ślady w wewnętrznej dynamice modelu językowego i że ślady te mogą być uchwycone przez interpretowalne cechy geometryczne oraz probabilistyczne wyprowadzane ze stanów ukrytych i rozkładów prawdopodobieństwa. W tym celu opracowano potok badawczy obejmujący ekstrakcję ośmiu cech, budowę macierzy cech, uczenie klasyfikatora i ewaluację na zbiorach TriviaQA, SQuAD oraz NQOpen dla modelu Llama-3.1-8B.

5.1. Realizacja celów pracy

Opracowano i zaimplementowano potok ekstrakcji cech wewnętrznych z modelu językowego podczas pojedynczego przebiegu generowania odpowiedzi. Potok obejmuje zapis stanów ukrytych i logitów z wybranych warstw, obliczenie ośmiu cech tokenowych, konstrukcję macierzy cech Φ oraz jej agregację do postaci wymaganej przez klasyfikator. Wyniki AUROC wyraźnie powyżej poziomu losowego we wszystkich konfiguracjach wskazują, że badane cechy niosą sygnał predykcyjny. Najlepszy wynik, AUROC = 0,897, uzyskała regresja logistyczna dla modelu Llama-3.1-8B na zbiorze TriviaQA (tabela 4.1).

Analiza pojedynczych cech (sekcja 4.5 w rozdziale 4) wskazuje, że `layer_drift` jest najsilniejszym pojedynczym predyktorem (0,884), a wysoką wartość predykcyjną wykazują również `unc_layer_drift` (0,858), entropie (0,848) i `unc` (0,839). Najniższą siłę predykcyjną spośród wszystkich cech wykazuje `context_dist` oraz `prediction_spread` (0,815), przy czym rozstęp między najlepszą a najslabszą cechą wynosi zaledwie 0,069, co sugeruje, że każdy z badanych sygnałów wewnętrznych dobrze radzi sobie z wykrywaniem pseudohalucynacji.

Badanie skuteczności metody na danych z różnych zbiorów (rysunek 4.1) wskazuje na względną stabilność wytrenowanego klasyfikatora, przy jednoczesnym występowaniu spadków skuteczności w części konfiguracji. Największy spadek zaobserwowano dla modelu trenowanego na NQOpen i testowanego na SQuAD (AUROC = 0,746), natomiast w kierunku NQOpen → TriviaQA uzyskano wynik (0,860) przewyższający skuteczność osiąganą w ramach testu na zbiorze, na którym przeprowadzono trening (NQOpen).

5.2. Odpowiedzi na pytania badawcze

Pierwsze z postawionych pytań badawczych dotyczyło tego, czy cechy wewnętrzne modelu pozwalają klasyfikować odpowiedzi powyżej poziomu losowego. Wyniki AUROC bez transferu między zbiorami mieszczą się w przedziale od 0,831 do 0,897 w zależności od zbioru danych (tabela 4.1), co wskazuje, że badane cechy niosą użyteczny sygnał predykcyjny. W teście generalizacji krzyżowej wyniki sięgają do 0,746 (SQuAD → NQOpen).

Drugie pytanie dotyczyło tego, które spośród badanych cech wnoszą największy wkład w wynik klasyfikacji. Analiza cech pojedynczych wskazuje na `layer_drift` jako najsilniejszy predyktor, jednak żadna z cech nie dominuje bezwzględnie, a pełny zestaw cech przewyższa każdą z nich rozpatrywaną osobno, co sugeruje komplementarność sygnałów wewnętrznych wykrywanych przez poszczególne cechy.

Trzecie zagadnienie sprawdzało, czy metoda zachowuje skuteczność przy zmianie zbioru danych. Uzyskane wyniki wykazują, że zaproponowane cechy zachowują zdolność generalizacji między różnymi zbiorami danych. Klasyfikator wytrenowany na jednym zbiorze zachowuje skuteczność na pozostałych, lecz z widocznymi spadkami w niektórych konfiguracjach, zwłaszcza przy transferze między zbiorami SQuAD i NQOpen.

5.3. Ograniczenia

Wyniki pracy należy interpretować z uwzględnieniem kilku istotnych ograniczeń. Etykiety wyznaczone przez BLEURT i ROUGE-L stanowią jedynie przybliżenie poprawności odpowiedzi. Ponadto eksperymenty ograniczają się do stosunkowo niewielkiej części trzech zbiorów pytań i odpowiedzi w języku angielskim oraz do jednego modelu, co oznacza, że wyniki mogą się różnić dla innych języków, modeli czy zbiorów danych. Wyniki AUROC pochodzą z jednego podziału danych. Eksperymenty przeprowadzono na pierwszych 10 000 przykładach z każdego zbioru według kolejności na platformie Hugging Face. Powtórzenie eksperymentów na próbie losowej pozwoliłoby ocenić, czy kolejność przykładów wpływa na uzyskane wyniki.

5.4. Kierunki dalszych badań

Naturalne rozszerzenia niniejszej pracy obejmują kilka kierunków. Aby umożliwić rzetelne porównanie wyników, wskazane byłoby przeprowadzenie ewaluacji metod referencyjnych, takich jak LapEigvals czy podejścia oparte na perplexity, na tych samych zbiorach danych i w tych samych warunkach eksperymentalnych, co proponowana metoda. Obecne porównanie z literaturą jest utrudnione ze względu na różnice w protokołach ewaluacji stosowanych przez poszczególnych autorów. Rozszerzenie eksperymentów na większą liczbę próbek z każdego zbioru oraz dodatkowe modele pozwoliłoby ponadto ocenić, czy wyekstrahowane cechy uchwytują sygnał niezależny od konkretnej architektury. Istotnym kierunkiem jest również rozwiązanie problemu jakości etykiet. Zastąpienie automatycznego etykietowania opartego na BLEURT i ROUGE-L ręczną adnotacją, choćby dla wybranej próbki danych, pozwoliłoby ocenić rzeczywisty poziom błędów etykietowania i ich wpływ na wyniki klasyfikacji.

Bibliografia

- [1] Llama Team. The llama 3 herd of models. *CoRR*, abs/2407.21783, 2024. URL: <https://doi.org/10.48550/arXiv.2407.21783>, [arXiv:2407.21783](https://arxiv.org/abs/2407.21783), [doi:10.48550/ARXIV.2407.21783](https://doi.org/10.48550/ARXIV.2407.21783).
- [2] Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In Regina Barzilay and Min-Yen Kan, editors, *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, ACL 2017, Vancouver, Canada, July 30 - August 4, Volume 1: Long Papers*, pages 1601–1611. Association for Computational Linguistics, 2017. URL: <https://doi.org/10.18653/v1/P17-1147>, [doi:10.18653/V1/P17-1147](https://doi.org/10.18653/V1/P17-1147).
- [3] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. Squad: 100, 000+ questions for machine comprehension of text. *CoRR*, abs/1606.05250, 2016. URL: <http://arxiv.org/abs/1606.05250>, [arXiv:1606.05250](https://arxiv.org/abs/1606.05250).
- [4] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association of Computational Linguistics*, 7:453–468, 2019. [doi:10.1162/tacl_a_00276](https://doi.org/10.1162/tacl_a_00276).
- [5] Thibault Sellam, Dipanjan Das, and Ankur P. Parikh. BLEURT: learning robust metrics for text generation. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel R. Tetreault, editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*, pages 7881–7892. Association for Computational Linguistics, 2020. URL: <https://doi.org/10.18653/v1/2020.acl-main.704>, [doi:10.18653/V1/2020.ACL-MAIN.704](https://doi.org/10.18653/V1/2020.ACL-MAIN.704).
- [6] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain, July 2004. Association for Computational Linguistics. URL: <https://aclanthology.org/W04-1013/>.
- [7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pages 5998–6008, 2017. URL: <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.

- [8] Grant Sanderson. Attention in transformers, visually explained, 2024. URL: <https://www.3blue1brown.com/lessons/attention/>.
- [9] Tomás Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. In Yoshua Bengio and Yann LeCun, editors, *1st International Conference on Learning Representations, ICLR 2013, Scottsdale, Arizona, USA, May 2-4, 2013, Workshop Track Proceedings*, 2013. URL: <http://arxiv.org/abs/1301.3781>.
- [10] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Comput. Surv.*, 55(12):248:1–248:38, 2023. doi:[10.1145/3571730](https://doi.org/10.1145/3571730).
- [11] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Trans. Inf. Syst.*, 43(2):42:1–42:55, 2025. doi:[10.1145/3703155](https://doi.org/10.1145/3703155).
- [12] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pages 9004–9017. Association for Computational Linguistics, 2023. URL: <https://doi.org/10.18653/v1/2023.emnlp-main.557>, doi:[10.18653/V1/2023.EMNLP-MAIN.557](https://doi.org/10.18653/V1/2023.EMNLP-MAIN.557).
- [13] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nat.*, 630(8017):625–630, 2024. URL: <https://doi.org/10.1038/s41586-024-07421-0>, doi:[10.1038/S41586-024-07421-0](https://doi.org/10.1038/S41586-024-07421-0).
- [14] Amos Azaria and Tom M. Mitchell. The internal state of an LLM knows when it’s lying. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023, Singapore, December 6-10, 2023*, Findings of ACL, pages 967–976. Association for Computational Linguistics, 2023. URL: <https://doi.org/10.18653/v1/2023.findings-emnlp.68>, doi:[10.18653/V1/2023.FINDINGS-EMNLP.68](https://doi.org/10.18653/V1/2023.FINDINGS-EMNLP.68).
- [15] Weihang Su, Changyue Wang, Qingyao Ai, Yiran Hu, Zhijing Wu, Yujia Zhou, and Yiqun Liu. Unsupervised real-time hallucination detection based on the internal states of large language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024*, Findings of ACL, pages 14379–14391. Association for Computational Linguistics, 2024. URL: <https://doi.org/10.18653/v1/2024.findings-acl.854>, doi:[10.18653/V1/2024.FINDINGS-ACL.854](https://doi.org/10.18653/V1/2024.FINDINGS-ACL.854).

- [16] Junjie Hu, Gang Tu, ShengYu Cheng, Jinxin Li, Jinting Wang, Rui Chen, Zhilong Zhou, and Dongbo Shan. HARP: hallucination detection via reasoning subspace projection. *CoRR*, abs/2509.11536, 2025. URL: <https://doi.org/10.48550/arXiv.2509.11536>, [arXiv:2509.11536](https://arxiv.org/abs/2509.11536), [doi:10.48550/ARXIV.2509.11536](https://doi.org/10.48550/ARXIV.2509.11536).
- [17] Jakub Binkowski, Denis Janiak, Albert Sawczyn, Bogdan Gabrys, and Tomasz Kajdaniowicz. Hallucination detection in llms using spectral features of attention maps. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 24354–24385. Association for Computational Linguistics, 2025. URL: <https://doi.org/10.18653/v1/2025.emnlp-main.1239>, [doi:10.18653/V1/2025.EMNLP-MAIN.1239](https://doi.org/10.18653/V1/2025.EMNLP-MAIN.1239).
- [18] Alexandra Bazarova, Andrei Volodichev, Aleksandr Yugay, Andrey Shulga, Alina Ermilova, Konstantin Polev, Julia Belikova, Rauf Parchiev, Dmitry Simakov, Maxim Savchenko, Andrey Savchenko, Serguei Barannikov, and Alexey Zaytsev. Hallucination detection in llms with topological divergence on attention graphs, 2026. URL: <https://arxiv.org/abs/2504.10063>, [arXiv:2504.10063](https://arxiv.org/abs/2504.10063).
- [19] Jie Ren, Jiaming Luo, Yao Zhao, Kundan Krishna, Mohammad Saleh, Balaji Lakshminarayanan, and Peter J. Liu. Out-of-distribution detection and selective generation for conditional language models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. URL: <https://openreview.net/forum?id=kJUS5nD0vPB>.
- [20] Aman Sharma and Paras Chopra. Think just enough: Sequence-level entropy as a confidence signal for LLM reasoning. *CoRR*, abs/2510.08146, 2025. URL: <https://doi.org/10.48550/arXiv.2510.08146>, [arXiv:2510.08146](https://arxiv.org/abs/2510.08146), [doi:10.48550/ARXIV.2510.08146](https://doi.org/10.48550/ARXIV.2510.08146).
- [21] Sewon Min, Kalpesh Krishna, Xinxu Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. Factscore: Fine-grained atomic evaluation of factual precision in long form text generation. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pages 12076–12100. Association for Computational Linguistics, 2023. URL: <https://doi.org/10.18653/v1/2023.emnlp-main.741>, [doi:10.18653/V1/2023.EMNLP-MAIN.741](https://doi.org/10.18653/V1/2023.EMNLP-MAIN.741).
- [22] Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu, Kashun Shum, Randy Zhong, Juntao Song, and Tong Zhang. Ragtruth: A hallucination corpus for developing trustworthy retrieval-augmented language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, pages 10862–10878. Association for Computational

Linguistics, 2024. URL: <https://doi.org/10.18653/v1/2024.acl-long.585>, doi:10.18653/V1/2024.ACL-LONG.585.

- [23] Shahul ES, Jithin James, Luis Espinosa Anke, and Steven Schockaert. Ragas: Automated evaluation of retrieval augmented generation. In Nikolaos Aletras and Orphée De Clercq, editors, *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics, EACL 2024 - System Demonstrations, St. Julians, Malta, March 17-22, 2024*, pages 150–158. Association for Computational Linguistics, 2024. URL: <https://aclanthology.org/2024.eacl-demo.16>.
- [24] Xuefeng Du, Chaowei Xiao, and Sharon Li. Haloscope: Harnessing unlabeled LLM generations for hallucination detection. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL: http://papers.nips.cc/paper_files/paper/2024/hash/ba92705991cfbbcedc26e27e833ebbae-Abstract-Conference.html.
- [25] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake VanderPlas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Edouard Duchesnay. Scikit-learn: Machine learning in python. *J. Mach. Learn. Res.*, 12:2825–2830, 2011. URL: <https://dl.acm.org/doi/10.5555/1953048.2078195>, doi:10.5555/1953048.2078195.
- [26] Kiho Park, Yo Joong Choe, and Victor Veitch. The linear representation hypothesis and the geometry of large language models. In Ruslan Salakhutdinov, Zico Kolter, Katherine A. Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, Proceedings of Machine Learning Research, pages 39643–39666. PMLR / OpenReview.net, 2024. URL: <https://proceedings.mlr.press/v235/park24c.html>.
- [27] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. URL: <https://transformer-circuits.pub/2021/framework/index.html>.
- [28] James A Hanley and Barbara J McNeil. The meaning and use of the area under a receiver operating characteristic (roc) curve. *Radiology*, 1982. URL: <https://pubmed.ncbi.nlm.nih.gov/7063747/>.

Dodatek A

Materiały uzupełniające i deklaracje

Niniejszy rozdział zawiera szczegółowy opis struktury promptów stosowanych podczas generowania odpowiedzi, charakterystykę wykorzystanych zbiorów danych, wyniki eksperymentów uzasadniających dobór parametrów wewnętrznych dla wybranych cech oraz deklarację dotyczącą wykorzystania narzędzi sztucznej inteligencji.

A.1. Struktura promptów i charakterystyka zbiorów danych

W celu wszechstronnej ewaluacji metod detekcji pseudohalucynacji, eksperymenty przeprowadzono na trzech zróżnicowanych zbiorach danych. Każdy z nich testuje odmienne aspekty generowania tekstu oraz nakłada inne ograniczenia na przestrzeń informacyjną modelu.

A.1.1. Zbiór TriviaQA

TriviaQA to zbiór danych zawierający pary pytań i odpowiedzi pochodzących z quizów ciekawostkowych. Charakteryzują się one relatywnie wysokim poziomem trudności i wymagają od modelu szerokiej wiedzy ogólnej (tzw. wiedzy parametrycznej).

Prompt wejściowy dla tego zbioru konstruowany jest według następującego wzorca:

```
"Question: {pytanie}\nAnswer briefly:"
```

Pytania w TriviaQA mają z reguły jednoznaczną, krótką odpowiedź faktograficzną, co czyni ten zbiór szczególnie odpowiednim do badania pseudohalucynacji zewnętrznych, ponieważ model musi odwołać się wyłącznie do wiedzy zakodowanej w parametrach, bez żadnego kontekstu pomocniczego.

A.1.2. Zbiór SQuAD

SQuAD (Stanford Question Answering Dataset) to klasyczny zbiór zadań z obszaru czytania ze zrozumieniem. Odpowiedź na pytanie stanowi w nim bezpośredni fragment tekstu z powiązanego artykułu z Wikipedii.

Pełny szablon dla tego zadania przyjmuje postać:

```
"Context: {kontekst}\nQuestion: {pytanie}\nAnswer briefly:"
```

Wprowadzenie jawnego bloku Context diametralnie zmienia dynamikę pracy modelu. Eksperyment na tym zbiorze pozwala zbadać, jak zachowują się stany ukryte w scenariuszu *grounded generation* (generowania zakotwiczonego w tekście źródłowym). Umożliwia to odróżnienie pseudohalucynacji wynikających z niewiedzy modelu od pseudohalucynacji wynikających z błędów uwagi (gdy model ignoruje podany kontekst).

A.1.3. Zbiór NQOpen

NQOpen (Natural Questions Open) bazuje na rzeczywistych, otwartych zapytaniach wpisywanych przez użytkowników do wyszukiwarki Google. Zadaniem modelu jest udzielenie krótkiej odpowiedzi bez dostępu do dokumentów źródłowych.

Szablon dla tego zbioru danych wygląda następująco:

```
"Question: {pytanie}\nAnswer briefly:"
```

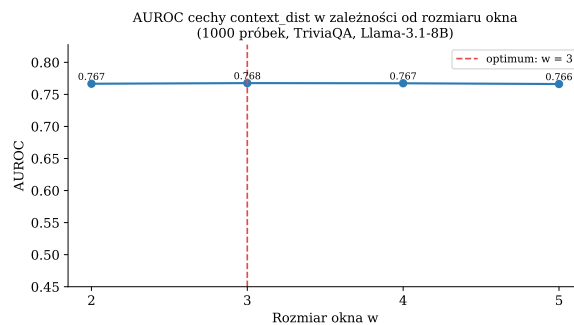
W przeciwieństwie do sztucznie przygotowanych quizów (TriviaQA), zapytania użytkowników w NQOpen są często sformułowane w sposób potoczny lub niejednoznaczny. Pytania w NQOpen odzwierciedlają rzeczywiste potrzeby informacyjne użytkowników, co pozwala zbadać zachowanie cech wewnętrznych modelu w warunkach bliższych praktycznemu zastosowaniu niż klasyczny format quizowy.

A.2. Dobór parametrów wewnętrznych cech

Dla cech `context_dist` oraz `prediction_spread` przeprowadzono dodatkowe eksperymenty optymalizacyjne na wydzielonych próbach walidacyjnych w celu wyznaczenia ich najefektywniejszych konfiguracji.

A.2.1. Rozmiar okna kontekstowego

W przypadku cechy `context_dist` (mierzącej dystans kontekstowy w reprezentacjach) analizowano wpływ rozmiaru okna wstecznego (w). Na podstawie wyników uzyskanych na niewielkiej próbce danych wybrano wartość $w = 3$. Rysunek A.1 prezentuje zależność wskaźnika AUROC od wybranego rozmiaru okna. Okno o długości 3 tokenów zapewnia optymalny kompromis pomiędzy wychwytywaniem lokalnych zależności składniowych a eliminacją szumu informacyjnego z odległych pozycji.

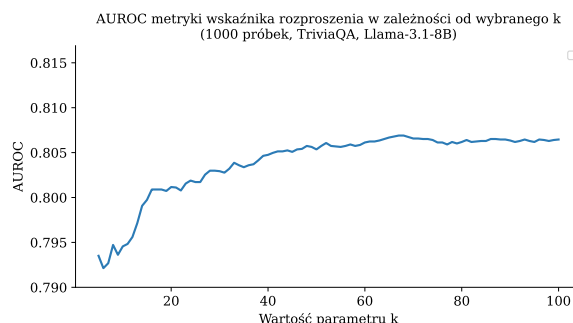


Rysunek A.1. Wybór rozmiaru okna kontekstowego dla cechy `context_dist`.

A.2.2. Rozmiar zbioru kandydatów

Dla cechy `prediction_spread` (szacującej rozproszenie predykcyjne) eksperymentalnie dobrano rozmiar zbioru kandydatów (K). Ostatecznie wybrano wartość $K = 50$. Dalsze zwiększanie tego parametru nie przynosiło statystycznie istotnych poprawek w zdolnościach

predykcyjnych modelu (rysunek A.2), generowało natomiast niepotrzebne obciążenie pamięciowe związane z przetwarzaniem bardzo długich ogonów rozkładu prawdopodobieństwa.



Rysunek A.2. Wybór rozmiaru zbioru kandydatów dla cechy `prediction_spread`.

A.3. Deklaracja wykorzystania sztucznej inteligencji

W procesie tworzenia niniejszej pracy zintegrowano zaawansowane narzędzia sztucznej inteligencji. Przegląd literatury przeprowadzono za pomocą serwisu Perplexity, a jej analizę i ekstrakcję kluczowych wniosków powierzono narzędziu NotebookLM. Do refaktoryzacji kodu, generowania wykresów oraz detekcji błędów w kodzie wykorzystano Claude Code, który wspomagał również redakcję tekstu. Ostateczna redakcja tekstu została przeprowadzona przy wsparciu modelu Gemini.